



 Latest updates: <https://dl.acm.org/doi/10.1145/3747346>

SURVEY

A Multivocal Review of MLOps Practices, Challenges and Open Issues

BEYZA EKEN, Sakarya University, Serdivan, Sakarya, Turkey

SAMODHA PALLEWATTA, The University of Adelaide, Adelaide, SA, Australia

NGUYEN TRAN, The University of Adelaide, Adelaide, SA, Australia

AYSE TOSUN, Istanbul Technical University, Istanbul, Turkey

MUHAMMAD ALI BABAR, The University of Adelaide, Adelaide, SA, Australia

Open Access Support provided by:

The University of Adelaide

Istanbul Technical University

Sakarya University



PDF Download
3747346.pdf
23 March 2026
Total Citations: 6
Total Downloads:
3220

Published: 08 September 2025

Online AM: 08 July 2025

Accepted: 13 June 2025

Revised: 05 June 2025

Received: 11 June 2024

[Citation in BibTeX format](#)

A Multivocal Review of MLOps Practices, Challenges and Open Issues

BEYZA EKEN, Sakarya University, Sakarya, Turkey

SAMODHA PALLEWATTA, The University of Adelaide, Adelaide, Australia

NGUYEN TRAN, The University of Adelaide, Adelaide, Australia

AYSE TOSUN, Istanbul Technical University, Istanbul, Turkey

MUHAMMAD ALI BABAR, The University of Adelaide, Adelaide, Australia

MLOps has emerged as a key solution to address many socio-technical challenges of bringing ML models to production, such as integrating ML models with non-ML software, continuous monitoring, maintenance, and retraining of deployed models. Despite the utility of MLOps, an integrated body of knowledge regarding MLOps remains elusive because of its extensive scope due to the diversity of ML productionalization challenges it addresses. Whilst the existing literature reviews provide valuable snapshots of specific practices, tools, and research prototypes related to MLOps at various times, they focus on particular facets of MLOps, thus fail to offer a comprehensive and invariant framework that can weave these perspectives into a unified understanding of MLOps. This article presents a Multivocal Literature Review that systematically analyzes a corpus of 150 peer-reviewed and 48 grey literature to synthesize a unified conceptualization of MLOps and develop a snapshot of its best practices, adoption challenges, and solutions.

CCS Concepts: • **Software and its engineering** → **Software development methods**; • **General and reference** → *Empirical studies; Surveys and overviews*;

Additional Key Words and Phrases: MLOps, machine learning operations, DevOps, CI/CD

ACM Reference Format:

Beyza Eken, Samodha Pallewatta, Nguyen Tran, Ayse Tosun, and Muhammad Ali Babar. 2025. A Multivocal Review of MLOps Practices, Challenges and Open Issues. *ACM Comput. Surv.* 58, 2, Article 39 (September 2025), 35 pages. <https://doi.org/10.1145/3747346>

1 Introduction

There is a huge trend of incorporating **artificial intelligence (AI)** and **machine learning (ML)** components into software systems to construct ML-enabled systems and take advantage of their well-established inferential and predictive capabilities. IBM Global AI Adoption Index for 2022 reports that AI/ML adoption is increasing rapidly, with 44% of the organizations working on

Beyza Eken and Samodha Pallewatta contributed equally to this research.

Authors' Contact Information: Beyza Eken, Sakarya University, Sakarya, Turkey; e-mail: beken@sakarya.edu.tr; Samodha Pallewatta, The University of Adelaide, Adelaide, South Australia, Australia; e-mail: skpallewatta92@gmail.com; Nguyen Tran, The University of Adelaide, Adelaide, South Australia, Australia; e-mail: nguyen.tran@adelaide.edu.au; Ayse Tosun, Istanbul Technical University, Istanbul, Turkey; e-mail: tosunay@itu.edu.tr; Muhammad Ali Babar, The University of Adelaide, Adelaide, South Australia, Australia; e-mail: ali.babar@adelaide.edu.au.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/09-ART39

<https://doi.org/10.1145/3747346>

embedding AI/ML in their products or processes in 2022 [10]. As of January 2023, 34% of the experimental models are registered as candidates for production, which is a significant increase compared to 20% in 2022 [2].

Despite the interest in ML-enabled systems, most organizations that build ML pilots fail to bring their ML models to production [19] due to novel operationalizing challenges of ML-enabled software systems compared to traditional software. *Productionalization of ML models* refers to the process of transitioning ML models from a laboratory setting to a production environment, where they must handle live traffic from real users and provide inference services reliably at scale while ensuring safety. Successful productionalization of ML models requires organizations to address multiple challenges, both technical and social in nature. For instance, one prominent issue is the integration of ML models into the surrounding software systems (PS142). To enable this integration, software engineers must be able to package the trained model (PS142) and implement the necessary data transformation logic to feed data from the surrounding software components to the ML model with correct syntax and semantics (PS179). Failure to do so could result in erroneous predictions, such as when feeding metric measurements to a model that expects imperial measurements. The details regarding syntax, semantics, and data transformation used by ML models might not be apparent or readily documented. Consequently, resolving this technical challenge requires software engineers to collaborate closely with data scientists and ML engineers despite these disciplines' differing languages and cultures (PS150).

Another common problem in the productionalization of ML models lies in the monitoring and maintaining ML-enabled software in production (PS107). The IT operations team must implement additional telemetry to monitor the behaviour of ML models, allowing them to detect when models suffer from concept drifts or violate safety constraints (PS151). Again, this task requires a skill set different from that typically found among IT operations teams. It necessitates collaboration between IT staff and data scientists with distinct disciplinary cultures and lexicon (PS150).

Additionally, retraining and updating models when problems such as drifts are detected in production represent another challenge in operating ML-enabled systems. The retraining process necessitates new data, likely sourced from the production environment. This data may contain sensitive personal information and must undergo appropriate preprocessing (PS151). Developing this data pipeline requires collaboration between software engineers, IT operations, data engineers, and data scientists. During the model training phase, it's critical to maintain provenance records of experiments for analysis and compliance purposes (PS153), (PS165). After a new model is created, software engineers and IT operators face the challenge of rolling out the new models quickly and safely and need to rely on strategies like A/B testing, canary deployments, and blue/green deployment (PS111), (PS168). Velocity and safety ultimately drive the success of ML model productionalization.

The concept of MLOps emerged as a key solution to address these challenges of ML model productionalization. MLOps was first proposed by the research community in 2015 to expedite ML lifecycle management while achieving the high scalability required by business applications [5, 21]. Research indicates that the effective operationalization of ML-enabled systems is contingent on employing robust MLOps practices, processes, and technologies [19]. A survey conducted by Databricks in 2023 indicates that, although the percentage of ML models reaching production remains low, there has been a significant increase with the rapid advancements in MLOps solutions (e.g., MLFlow, Hugging Face, SageMaker, Google Vertex AI).

Despite the widespread prevalence of MLOps, an evidence-based body of knowledge regarding MLOps practices and tools grounded in a unified conceptualization of the MLOps concept remains elusive for several reasons. First, the sheer scope of MLOps encompasses a broad array of practices, techniques, and corresponding technologies, complicating the effort to systematize the knowledge.

For example, MLOps can be viewed from the perspective of ML “operations” (i.e., bringing ML models from the lab into deployed software systems), leading to research related to the packaging of ML models, interoperability with surrounding software systems, and testing of ML-enabled systems (e.g., (PS179)). MLOps is also often conceptualized as “DevOps for ML”, leading to the development of practices and tools for defining and managing automated pipelines of ML lifecycle activities (e.g., (PS135), (PS172)). Additionally, MLOps is also viewed from the perspective of systematic management of ML experiments and related assets (e.g., datasets, feature sets, trained models), leading to practices such as applying version control to ML assets and recording experiments to support both analysis and provenance for compliance and security (e.g., [18, 20]). On-demand and continuous monitoring of model performance and safety in a production environment is yet another critical practice associated with MLOps (e.g., [24]). Finally, given that ML model productionalization involves collaboration among multiple teams, many MLOps best practices emphasize the need for organizational and cultural changes to facilitate necessary cross-team collaboration in handling ML productionalization (e.g., establishing project teams (PS153), (PS111), building specialized ML engineering teams).

The diversity of information sources on MLOps compounds the issue. Particularly, gray literature such as training materials and best practices from various platform providers (PS153), (PS165), (PS154), (PS106), (PS107) play a significant role in MLOps since it is inherently driven by practice. This gray literature must be compared and contrasted with peer-reviewed literature to synthesize a comprehensive and unbiased understanding of the field, which requires substantial time and effort. This challenge is exacerbated by the previously mentioned diversity in what constitutes “MLOps”. There is considerable confusion surrounding MLOps due to its complexity, diversity, and broad scope (PS154), creating obstacles for both practitioners and researchers navigating the current MLOps landscape, hindering effective adoption and subsequent improvements in MLOps practices and solutions.

In this article, our objective is to build an evidence-based knowledge base for MLOps based on a unified conceptualization of the MLOps concept by integrating input from the gray literature and peer-reviewed literature to address the challenges that practitioners and researchers face when working with MLOps. The unified conceptualization of MLOps in this context denotes a coherent and comprehensive model that accounts for the various points of view on MLOps with respect to its definition, activities carried out within its scope, and the roles and responsibilities involved. Such a conceptualization creates a framework for identifying and classifying the diverse practices, processes, and technologies under the MLOps umbrella. “Evidence-based” in this context signifies that the knowledge base and conceptualization will be synthesized inductively from the literature. This knowledge base’s benefit is providing practitioners with a framework for understanding, assessing, and adopting MLOps practices while offering researchers a roadmap for open research opportunities. We applied the Systematic **Multivocal Literature Review (MLR)** and thematic analysis methodologies to address the research goal. The MLR methodology enables us to systematically identify, select, and extract information from peer-reviewed and gray literature on MLOps published up to September 2023. The thematic analysis allows us to systematically analyze the extracted data and synthesize the conceptualization of MLOps, best practices, adoption challenges, and solutions to build a robust body of knowledge for researchers and practitioners interested in MLOps.

The multifaceted nature of MLOps and the abundance of primary studies classified under this umbrella have led to many literature reviews exploring various aspects of MLOps. For example, Steidl et al. [23] conducted a tertiary analysis that identified 33 reviews encompassing 1,397 primary studies on MLOps and found minimal overlap in the coverage of these reviews. Whilst the existing literature reviews provide valuable snapshots of specific practices, tools, and research prototypes

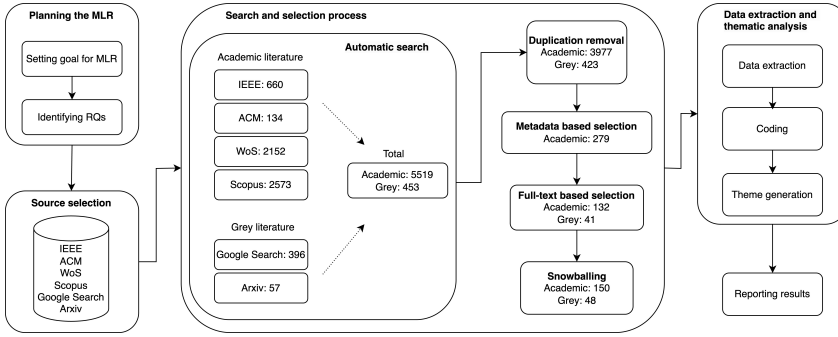


Fig. 1. Research methodology.

related to MLOps at various times, they focus on particular facets of MLOps (e.g., the DevOps angle, the experiment management angle, the continuous monitoring angle), thus fail to offer a **comprehensive and invariant** framework that can weave these perspectives into a unified understanding of MLOps. Such a framework is crucial because it aligns current and future practices and tools with their challenges. Although snapshots of MLOps technologies and practices can quickly become outdated as tools evolve, a robust framework allows for analyzing and categorizing present and future developments, paving the way for creating new snapshots. Thus, the systematic synthesis of a conceptual framework for MLOps from both peer-reviewed and gray literature represents our study's core focus and contribution (Section 3). Building upon this foundation, we identify and categorize the state-of-the-art practices and techniques for implementing MLOps, forming a "snapshot" of the literature (Section 4). Finally, we highlight practitioners' challenges when adopting MLOps and identify future research directions (Section 5).

2 Methodology

An MLR study allows researchers to collect and present information from the industry when the formal/academic literature is not sufficient [9]. Since MLOps is an emerging and practitioner-driven topic, only considering academic publications is not sufficient to cover all the knowledge related to this topic. Industry practitioners have been implementing and getting experience in MLOps to integrate ML-enabled systems into their software development pipelines, their knowledge is a crucial element in understanding MLOps. Thus, gray literature such as industrial white papers, blog posts, and company websites are the key sources for expanding the knowledge obtained from academic literature. Therefore, we believe an MLR would be the most suitable methodology.

We performed our MLR following the guidelines reported by Garousi et al. [9]. Consequently, our MLR consists of three main stages: (1) *planning the review*, (2) *conducting the review*, and (3) *reporting the review*. In the planning stage, we defined the **research questions (RQs)** and developed the protocol for identifying and selecting literature. In the conducting phase, we followed the specified protocol to identify and choose literature, extract data from the literature, and synthesize the extracted data to answer the RQs. We employed the Thematic Analysis methodology based on the guidelines proposed by Braun et al. [3] to carry out data synthesis. In the final stage, we write up the synthesis results. Figure 1 represents our MLR methodology.

2.1 Planning the Review

We designed three main RQs to systematically explore MLOps through academic and gray literature to address our research goals: (i) establish a foundational understanding of how MLOps is defined

Table 1. Research Questions Addressed in This MLR

Research Questions	Motivation
RQ1 - Conceptualization of MLOps	Exploring foundational aspects of MLOps is important for creating a common and standardized understanding of MLOps, optimizing workflows, enabling efficient adoption of MLOps practices across diverse contexts, and fostering collaboration.
RQ1.1 - How is MLOps defined in the literature?	RQ1.1 aims to provide insights on common and varying definitions of MLOps to facilitate better understanding of its foundational aspects to contribute towards creating a standardized and consistent usage within the community.
RQ1.2 - What are the primary activities and tasks that define an MLOps workflow?	RQ1.2 identifies the core components of MLOps, focusing on the essential activities and tasks within an MLOps workflow. Outlining these key elements, provides a foundation for process optimization and standardization, while offering practitioners actionable insights for efficient adoption and clarifying the operational impact of MLOps in their specific contexts.
RQ1.3 - What are the practitioner roles and responsibilities in MLOps?	RQ1.3 highlights the responsibilities of people who take part in MLOps settings. Establishing a common understanding of roles and their responsibilities leads to more efficient workflows and productive collaboration practices within the complex MLOps team structure.
RQ2 - What are the state-of-the-art best practices and techniques for implementing MLOps?	A complete view of how MLOps activities and tasks are performed in practice covering, frameworks, processes, tools and best practices to implement them is required for companies to plan for successful implementation of MLOps by minimizing the technical debt and improving the maintainability of the ML systems as they scale and grow in complexity over time. To this end, RQ2 aims to provide state-of-the-art MLOps practices and techniques covering social and technical aspects across the entire life cycle of ML models.
RQ3 - What are the challenges practitioners face when adopting MLOps, and what solutions have been proposed to address them?	Gaining a deeper understanding of the challenges practitioners face when implementing MLOps along with proposed solutions in the current literature, is crucial to see the bottlenecks in MLOps adoption. RQ3 addresses the state-of-the-art challenges that would be beneficial to practitioners to overcome those when they adopt MLOps for productionalizing their ML-enabled software systems. Further, existing challenges often lead to new and highly impactful research directions and innovative solutions.

Table 2. Search Strings

Academic literature	mlops OR "machine learning operations" OR mlsecops OR mlflow OR kubeflow OR cd4ml OR ((devops OR "continuous software engineering" OR cse OR "CI/CD" OR "continuous integration" OR "continuous deployment" OR "continuous delivery" OR "continuous test" OR "continuous verification" OR "continuous security" OR "continuous compliance" OR "continuous evolution" OR "continuous use" OR "continuous trust" OR "continuous monitoring" OR "continuous release" OR "continuous build" OR "continuous systems engineering") AND ("machine learning" OR "artificial intelligence" OR "deep learning" OR ml OR ai OR dl))
Gray literature	mlops OR "machine learning operations" OR (devops AND "machine learning") OR ("continuous software engineering" AND "machine learning")

and perceived, (ii) identify and map current practices, and (iii) identify challenges and solutions within the field. Table 1 outlines the RQs in detail and the motivations behind each.

2.2 Study Search and Selection Protocol

We based our protocol on the guidelines provided by Garousi et al. [9] for conducting MLR. Our protocol consists of three main steps: automatic search method using inclusive search strings, primary study selection by applying quality assessment and snowballing to identify further studies [26].

2.2.1 Automatic Search. For academic literature, we searched IEEEExplore Digital Library, ACM Digital Library, Web of Science, and Scopus using complex Boolean queries incorporating all related keywords into the search string to maximize the retrieval of relevant content from these databases. For gray literature, we used simpler, flexible search string as complex queries would limit the results of the search engine. We developed and evolved search strings through an exploratory search process, incorporating relevant keywords (e.g., continuous test, mlsecops) while avoiding restricting keywords (e.g., technologies, challenges, and tools) to capture relevant content. Based on this, we reached two different search strings for peer-reviewed and gray literature (Table 2). The search process was independently executed by two authors. Individual results were combined in a shared file, and non-overlapping items-potentially due to location-based differences-were reviewed and discussed for inclusion.

2.2.2 Primary Study Selection from Academic Literature. We removed those duplicated items from our pool of studies, which reduced the number of academic studies from 5519 to 3977. Afterwards, we applied a two-step study selection process to the academic literature: (1) metadata-based selection and (2) full text-based selection. The metadata-based selection is conducted by considering only the

paper title, paper keywords, and abstract of the paper, whereas the full-text selection is carried out by reviewing the full text of studies.

During the metadata-based selection, we followed the inclusion and exclusion criteria listed in our online appendix [7]. During the full text-based selection, we assessed the quality of each study considering the research aim, research design, methodology, results, limitations of the study, and the value added to research or practice by the study. A more detailed list of quality assessment criteria can be found in [7]. These criteria are adopted from [12] and [9], discussed and agreed by all authors.

At the beginning of both metadata and full-text-based selection steps, a pilot selection is carried out by all authors on a sample of the study pool. This pilot study is crucial to increase the inter-rater agreement and establish a common terminology among authors who later continue with study selection individually on a subset of relevant studies. Pilot study samples were picked randomly to avoid the bias that might occur during database search since the most relevant studies are naturally listed at the top of the search. We picked six studies for metadata-based selection and eight for full-text selection. These studies were reviewed by all the authors and the results were discussed.

After completing the pilot study, the study pool was split into three by considering the bias of study order, and the selection processes were carried out separately by three authors. When a decision cannot be made on a study by a single author, a second and if necessary a third reviewer assessed the same study, and the final decision was made accordingly. If authors were unable to reach a consensus, the study was added to the final list of primary studies in order to avoid the exclusion of any relevant content. We ended up having 279 studies after the metadata-based selection process. After reading the full text and assessing the quality of the studies based on our criteria, there were 132 academic studies left.

As the final step in academic literature selection, we applied forward and backward snowballing based on the guidelines by Wohlin et al. [26]. We received 39 more studies as a result of snowballing on the selected academic studies. Following the same inclusion and exclusion, and quality criteria we used in the prior steps, we included 18 out of the 39 studies found by snowballing, creating a final set of **150 peer-reviewed literature**.¹

2.2.3 Primary Study Selection from Gray Literature. First, we collected a list of studies by applying our search string on Google Search and Arxiv. We stopped our search on Google when the results reached theoretical saturation [9], i.e., meaning that no new content related to MLOps was found. The gathered results roughly covered 50% of all search results. At the end of the study collection, we had a total number of 453 studies including search engine results and pre-prints. Second, we applied the inclusion and exclusion criteria [7] to select primary studies. This involved removing video contents, non-free materials, and duplicates already identified through academic search. We ended up having 423 studies. Third, we applied our quality assessment criteria, reported in our online appendix [7], which are adopted from [9], on the remaining studies. We iteratively simplified the quality criteria provided by Garousi et al. [9] to ensure we captured all the relevant content. For example, we excluded many of the methodology related criteria as MLOps is still an emerging field and many are still trying to define it and its associated technologies and concepts. Hence studies tend to skip reporting experimental data or methodology. After the quality assessment, we selected studies that met at least seven out of nine criteria listed in the table. As the last step, a snowballing process was performed, leading to a final set of **48 gray literature**.¹

¹Primary studies are listed in our online appendix [7].

2.3 Data Extraction and Thematic Analysis

Thematic analysis is a qualitative research methodology widely used in **systematic literature reviews (SLRs)** to understand patterns within the extracted data by identifying, analyzing, and reporting themes [4, 6]. The thematic analysis begins with researchers familiarizing themselves with the data to gain a comprehensive understanding. Based on this understanding, they systematically create and assign *codes* to the data to signify interesting features and potential themes. The codes are then sorted into *themes*, which capture significant patterns in the data. Finally, themes are reviewed, accompanied by clear definitions and names for reporting. We followed the guidelines for thematic analysis established by Braun et al. [3] to carry out the analysis.

Understanding: The process commenced with all three authors engaging in data extraction to facilitate an in-depth understanding of the source material. We performed a pilot data extraction on six papers. To enhance inter-rater agreement, we discussed the results to refine the data extraction process and the predefined data-extraction form [9, 12]. Subsequently, data extraction was carried out separately, with the primary studies divided among the authors.

Coding: The coding process began after the completion of data extraction. We utilized a shared spreadsheet to maintain an up-to-date Code Book. Additionally, we maintained an Obsidian² vault to supplement the details in the Code Book within the spreadsheet. This vault contains a collection of notes, each providing a detailed description of a code along with related data extracted from the primary studies. These artefacts were shared and version-controlled, enabling authors to coordinate, validate the generated codes, and reuse codes when applicable. Each of the three authors focused on coding for one specific RQ, with biweekly catch-up meetings to synchronize our efforts.

Theme generation: We organized the generated codes into a hierarchy of themes and sub-themes. Each author proposed themes based on their assigned RQ, which were then collaboratively reviewed, and final themes were defined. For illustration, codes such as "Experiment Tracking", "Model Versioning", "Data Versioning", and "Model Tracking" were grouped under *Artifact Management*, capturing traceability concerns. Similarly, under the theme *MLOps Pipeline Implementation Related Challenges*, sub-themes such as "Optimized Pipeline Deployment" and "Managing Large Scale Pipelines" included codes like "Optimized Placement of Pipeline Modules", "Pipeline Management Scalability", "Data Quality Issues in Pipelines", and "Increased Model Resource Demand".

2.4 Threats to Validity

Potential threats to the validity of our study primarily concern the processes of study search, study selection, data extraction, and theme analysis. We collected studies from well-known academic databases and general search engines to ensure a comprehensive selection of the studies. Overview of selected studies and their distributions over time is available at our online appendix [7]. While going through search results, we excluded non-free resources, which is a limitation of our study. Additionally, we applied backward and forward snowballing techniques to identify potentially missed studies. We designed search strings to ensure broad coverage while avoiding restrictive keywords. Complex queries including many keywords were used in academic databases, while simpler queries were applied for gray literature to accommodate search engine limitations. This approach minimized the risk of omitting critical studies while maintaining relevance to MLOps. To ensure consistency in applying inclusion/exclusion criteria and quality assessment, we conducted pilot evaluations. All authors independently assessed randomly selected studies and discussed results to align on terminology and evaluation standards. In cases of uncertainty, decisions were made through consensus, ensuring reliability in the quality assessment process. We used a standardized template to extract key study details (e.g., objectives, methodologies, findings) [7]. A pilot extraction

²<https://obsidian.md/>

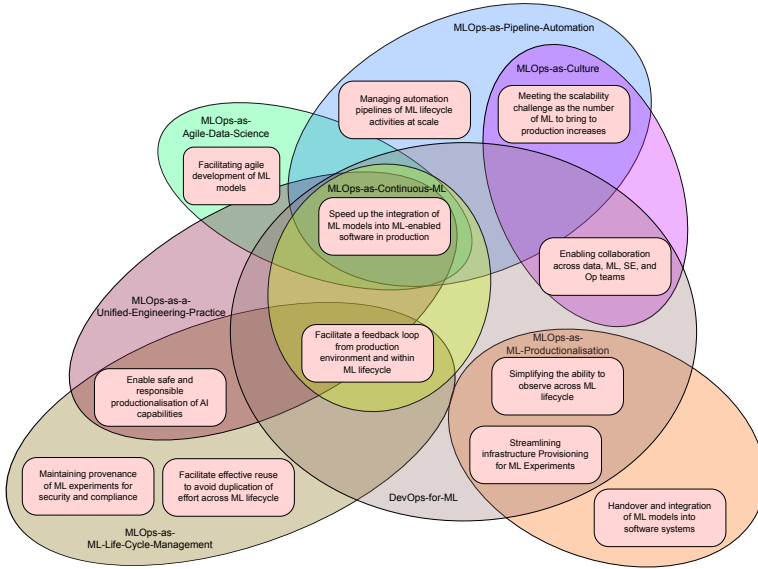


Fig. 2. MLOps definitions and the ML productionalization problems they address.

phase helped align understanding, with discrepancies resolved through discussion. For theme analysis, we employed an iterative coding process where three authors independently coded the data, resolving disagreements through discussion.

3 RQ1: Conceptualization of MLOps

3.1 RQ1.1 - How is MLOps Defined in the Literature?

Eight definitions for MLOps have been identified in the existing literature. These definitions can be considered the conceptual foundation for MLOps work, as they encompass what authors of the work perceive to be the primary challenges in ML productionalization, thereby delineating the scope of MLOps solutions (e.g., best practices, tools) to be proposed. The following subsections introduce the identified definitions, ranging from the most common to the least common. For example, 34 studies define MLOps as “continuous ML,” 32 refer to it as “DevOps for ML,” and 18 studies describe it as encompassing cultural and organizational practices for ML. We provide the complete statistics in our online appendix [7]. Figure 2 visualizes the relationship between eight definitions based on the ML productionalization problems on which they focus.

3.1.1 MLOps-as-Continuous-ML. This definition considers **MLOps as the integration of continuous engineering practices (e.g., Continuous Integration (CI) and Continuous Delivery (CD)) into the lifecycle of ML-enabled software systems**. The goal of MLOps-as-Continuous-ML is to enable rapid development and operations through continuous practices, allowing for the swift adaptation of existing solutions to changing conditions (e.g., business needs, concept shifts) (PS3), (PS24), (PS102). Moreover, continuous practices facilitate feedback loops between different phases critical in ML projects due to the experimental nature of ML engineering. For example, ML models are often updated after the initial monitoring phase, and feature engineering alongside model serving tasks are repeated to accommodate varying model needs (PS10).

The concept of MLOps-as-Continuous-ML is inspired by the well-known software engineering practice of CI/CD (continuous integration and delivery), which aims to integrate software to

deliver high-quality products frequently (see Appendix [7]), particularly emphasizing the parallels between delivering software to production and delivering ML to production. **Continuous training (CT)** (PS80) is often introduced as the ML-specific element. MLOps-as-Continuous-ML integrates continuous practices into many phases of the ML lifecycle (PS95), including model design (PS5), model training (PS1), serving (PS67), validating (PS86), monitoring (PS5), model delivery (PS28), and the integration of pipelines (PS102).

3.1.2 DevOps-for-ML. This definition considers MLOps as **an extension of DevOps practices for ML**. It should be noted that while continuous engineering practices are a key and perhaps the most well-known element of DevOps, the scope of DevOps itself extends beyond this to encompass practices such as streamlining infrastructure provisioning via **Infrastructure-as-Code (IaC)**, facilitating data-driven decision-making by providing developers with on-demand observability, as well as fostering the cultural shifts necessary to enable these cross-silo activities. These cultural shifts might include promoting collaboration between development and operations teams, encouraging a mindset of continuous improvement, and embracing automation. MLOps originates from the DevOps concept (PS1), (PS29), (PS32), (PS140) and targets the development (dev) and operationalization (ops) of ML systems (PS30), (PS88), (PS102). MLOps adopt practices and cultural aspects associated with DevOps (PS32), (PS24), (PS140), aiming to standardize and streamline the development, deployment, operation, and management of ML systems (PS108), (PS4). MLOps unifies the development and operational cycles carried out by different roles, ensuring the automated, frequent, and CD of high-performing ML systems (PS67).

3.1.3 MLOps-as-Pipeline-Automation. This definition considers MLOps as **the practice of automating the repetitive activities in the ML lifecycle by constructing and managing automated pipelines**. Details regarding ML lifecycle activities would be discussed in Section 3.2. Some benefits of automation include more rapid development and deployment (PS28) and easier management of the infrastructure to which ML models are deployed (PS72). MLOps advocates for automation (PS102) at all stages of ML system development and operations, including activities such as data collection, model building and deployment, and monitoring (PS1), (PS17), (PS30). Automation simplifies deployment by providing a more straightforward release and environment configuration process, allowing researchers to focus more on model optimization (PS44).

3.1.4 MLOps-as-Culture. Developing and operationalizing an ML-enabled system necessitates collaboration between many different roles, e.g., business analysts, ML engineers, and software engineers. As the number of ML applications and their complexity grow in the community, the need for establishing a standard for new roles and responsibilities and organizational collaboration practices becomes more prevalent. This definition considers MLOps as **the set of tools and managerial practices that lead to more sustainable development and operationalization of ML-enabled systems** (PS70) through enabling collaboration across diverse teams, e.g., data science team and IT professionals (PS45).

3.1.5 MLOps-as-Agile-Data-Science. As emphasized by several studies (PS81), (PS3), (PS86), (PS96), MLOps shares a similar philosophy with DevOps, but has unique aspects related to data and model. Further, MLOps is closely linked with agility in data science (PS30), (PS32), (PS36), (PS83). Specifically, producing and releasing ML models iteratively is the application of agile principles [1] in ML projects. Hence, this definition describes MLOps as **the intersection between data engineering, data science, ML, and operation to facilitate an agile end-to-end practice**.

3.1.6 MLOps-as-ML-Productionalization. This definition considers MLOps as **the process of operating ML models, which involves bringing them into a production environment**

(PS7). The purpose of MLOps from this perspective is to minimize the gap between experimental development and the production environment (PS163), as well as to facilitate the transfer of developed ML models to production at scale by ensuring reproducibility, reliability, and efficiency (PS71), (PS23). Research based on this definition of MLOps focuses on the processes, standards, and tools for packaging and integrating ML models into software systems, as well as on streamlined workflows from development to operationalization (PS22), (PS41), (PS89), (PS96).

3.1.7 MLOps-as-ML-Life-Cycle-Management. This definition considers MLOps as **the set of practices and tools to manage lifecycle activities of MLOps and their corresponding artefacts to ensure reproducibility, stability, optimization, and scalability**. One focus of MLOps research from this perspective is the handling of triggers, loops, and common artefacts that link various MLOps lifecycle activities (PS61), such as managing multiple versions of experiments, datasets, models, and metadata related to models (PS24). Another focus is the management, standardization and optimization of workflows across lifecycle activities (PS91), (PS94), (PS95). Another focus is permeating the lifecycle with responsible practices such as fairness of predictions (PS151).

3.1.8 MLOps-as-a-Unified-Engineering-Practice. Several studies define MLOps as **unifying ML-enabled systems' development, deployment, and operations**. This definition refers to supporting practices for developing (PS89), deploying (PS102), monitoring (PS25), and scaling (PS86) ML models quickly and efficiently, and managing their infrastructure configuration complexity (PS90) to ensure ML-enabled systems operate optimally and securely (PS90) within the predefined business thresholds (PS86).

3.2 RQ1.2 - What Are the Primary Activities and Tasks of MLOps?

Figure 3 shows a comprehensive MLOps pipeline that includes core MLOps activities and roles associated with them. We elaborate more on the activities in this section and roles in Section 3.3.

3.2.1 ML Project Initiation. ML project initiation mainly includes activities such as business analysis, domain understanding, project planning, specifying system requirements, and ML system design. Many stakeholders take part in these activities, i.e., business analysts, domain experts, data engineers and scientists, and ML and MLOps engineers. Specification of business and system requirements is crucial for planning and execution of an ML project, and successful deployment and operation of the ML system (PS105), (PS111). The process begins with formulating the business problem to establish a clear objective (PS41), helping assess business value (PS111), determine the feasibility of engineering an ML system (PS153). Further steps include obtaining security objectives of the company, e.g., confidentiality and privacy requirements of the data used through ML system (PS105), and regulatory rules (PS99). Moreover, ML system planning incorporates software project management knowledge areas [25] such as defining project scope (PS90), specifying ML use cases (PS70), designing model architecture, pipeline, and infrastructure (PS90), (PS70), identifying performance metrics for model evaluation (PS111), and establishing monitoring strategies with feedback mechanisms for business owners (PS70).

3.2.2 ML Lifecycle Activities. **Data collection** is the first step in the ML pipeline, involving the **acquisition of raw data** that will be processed in later steps to train an ML model. Often, raw data is gathered from relevant data sources, e.g., sensors (PS24), GitHub code repositories (PS27), or cameras (PS34). Some studies have more exploratory data collection steps. For example, data collection is combined with **data discovery** and **data selection**, i.e., collecting data for a drug discovery task requires performing experiments to produce data, followed by selection of data by domain experts to be used in later phases (PS37). Alternatively, some cases follow a data management plan prepared by data engineers to collect data to be fed into the ML pipeline

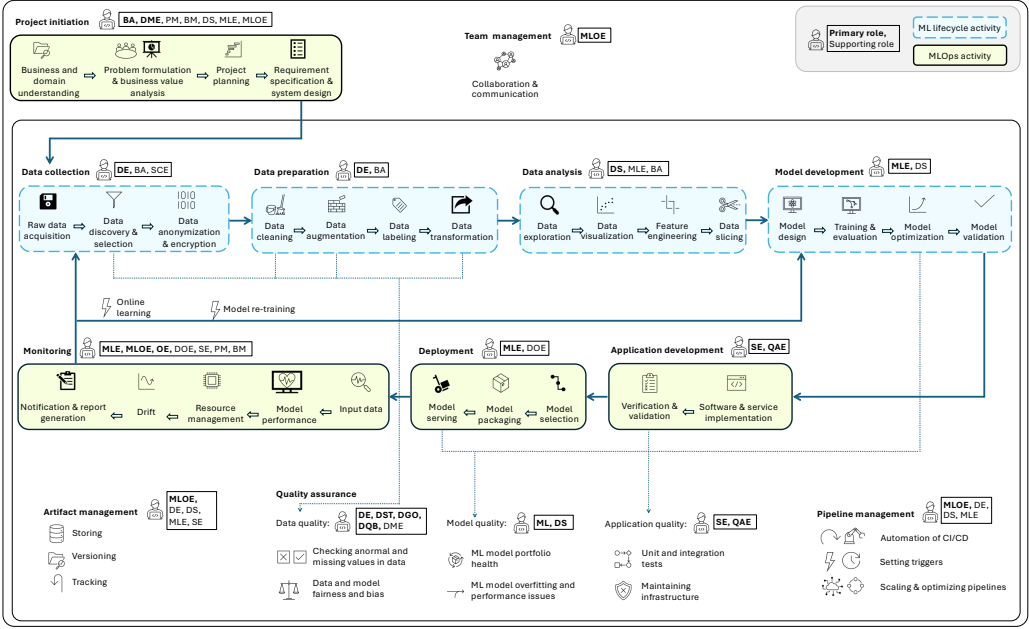


Fig. 3. A pipeline for ML development and operations, and associated roles: Domain Experts (DME); Business Analysts (BA); Data Engineers (DE); Data Scientists (DS); ML Engineers (MLE); MLOps Engineers (MLOE); DevOps Engineers (DOE); Software Engineers (SE); Quality Assurance Engineers (QAE); Operational Engineers (OPE); Technical and Solution Architects (ARC); Security Experts (SE); Business Owners/Managers (BM); Product Owners/Managers/Experts (PM); Data Governance Officer (DGO), Data Quality Board (DQB), Data Steward (DST)

(PS116). For security-critical systems, data collection process is applied with **anonymization and encryption** of the collected information to ensure data privacy (PS31).

Data preparation aims to process raw data for ML model building through cleaning the raw data, augmentation, labeling data points, transformation, and feature engineering. **Data cleaning** involves detecting and fixing errors in raw data, such as removing duplicates (PS24), filtering invalid and irrelevant data points (PS27), imputing missing values in data (PS37). This step is also known as **data filtering** in the literature (PS27). **Data augmentation** increases sample size to address limited data availability, with techniques like oversampling and simulation (PS5). **Data labeling** marks data points according to observations in the problem domain, involves data engineers, data scientists, and domain experts (PS116). While data labeling is a manual task in many cases, tools also support automated labeling (PS111). **Data transforming** adjusts the data format according to the needs of the ML system to be built, for example, removing characters for compatibility with other file systems (PS27). It also includes data representation changes, like distributional transformations on numerical data and data discretization for smaller units (PS31).

Data analysis includes **data exploration** based on data statistics, e.g., the mean and standard deviation, and quantiles (PS30), (PS62) to better understand the data before model development. It helps to understand and assess which features (PS112) and data slices (PS30) best meet ML model requirements. Data analysis also includes **data visualization** providing dashboards to facilitate the team's evaluations of data (PS25), (PS42), (PS114), (PS109). Moreover, data analysis supports data validation, which assesses the health of data by identifying anomalies or unexpected properties that could impact model quality (PS62). **Feature engineering** processes features to enhance model

performance and accuracy (PS64), (PS133). It includes sub-steps such as feature transformation, feature selection, feature extraction, feature construction, and feature storage (PS162), (PS114), (PS4), (PS102). Feature engineering is mostly seen as the responsibility of data engineers and scientists but also uses the expertise of other roles such as security (PS105) and product experts (PS31). Another mentioned activity is **data slicing** (PS136), which is the process of finding the subset of data that performs according to the use case. It overlaps with model building and evaluation steps since it explores the interest in data through evaluating the ML model's prediction performance on different data slices.

Model development is a core step in ML life-cycle, focused on constructing ML models. It includes model design, training, evaluation, selection, validation, and re-training. Model building partially overlaps with data analysis as it often involves exploring various training approaches, i.e., ML algorithm and hyperparameter configurations (PS114). **Model design** refers to designing and selecting the appropriate architecture and tools for ML model construction to ensure successful model training and prediction performance. Examples include designing neural network architectures (PS39), choosing the right algorithm for the problem (PS24), and optimizing ensemble models (PS31). **Model training** builds the ML model for prediction, using a learning algorithm to train it on the prepared data (PS111), (PS103). **Model evaluation** assesses predictive performance of the ML model, by testing it with a held-out test set, which includes a separate set of data points not used in training (PS29). Prediction performance is measured using relevant metrics for the use case (PS119). **Model optimization** improves the model's prediction performance. Hyperparameter tuning is a common method applied (PS24), (PS102). **Model validation** evaluates the model based on business goals (PS70), quality standards (PS62), and regulatory requirements (PS110). The common method for validating the quality of an ML model is comparing its predictive performance against a baseline model or a fixed threshold value (PS62), (PS157), (PS102).

3.2.3 Application Development. step addresses the implementation of software and/or service that will be interfaced with the deployed ML solution (PS108), (PS135). Additionally, this step involves verification, validation, and quality assurance activities of the developed application such as unit testing, integration testing, refactoring, maintenance to ensure reliability, functionality, and seamless interaction with the ML model (PS116), (PS123).

3.2.4 Deployment. The deployment phase integrates trained ML models into production environments, making model predictions accessible to users. Depending on the use case, deployment is achieved by integrating the ML model into a software system (PS116), serving it as a service (PS14), or creating containerized images of ML models (PS140). Deployment phase differs from routine ML activities (PS142), as it involves transferring the model to another team or infrastructure (PS110). As Neghawi et al. (PS116) note, "the deployment stage is a pivotal moment in the MLOps process," due to the transition from an experimental (ML development) to an actualized environment (ML production). Key deployment activities include **model selection** for choosing the most appropriate model among various models built for the same ML system, i.e., the best performing one (PS180), or based on client's requirements (PS141). Deployment of a model is decided based on model validation results, which often involve supervision of a manager (PS34), and collaboration of data scientists, business stakeholders, and ML engineers (PS114). Another key activity is **model packaging** for reproducibility and sharing (PS24), creating containerized model images with dependencies for inference (PS70), and selecting deployment methodologies based on the target ML system (PS24), (PS126). **Model serving** refers to making the ML model accessible for generating new predictions. A common approach is embedding the model within an application for user queries (PS29). Many studies favor the *Model-as-a-Service* paradigm, where models are exposed through REST API endpoints for scalability and ease of access (PS29), (PS140). Deployment requires collaboration

from ML and MLOps engineers, as well as software engineers, to ensure smooth integration into a production environment.

3.2.5 Monitoring. Monitoring is a key MLOps activity applied by many studies (PS26), (PS32) and supported by many tools (PS151), (PS114), (PS109), (PS147). It helps improve system health by providing feedback, often triggering the repetition of relevant MLOps activities. Ensuring operational stability and quality of an ML system in production is done by continuously observing data and model changes, model performance, system resources, application and its dependencies to capture and report the issues in a timely manner (PS144), (PS116). Input data changes, data and model biases, and concept drifts can be detected setting pre-defined thresholds and rules on data (PS111), or monitoring the **model performance** (PS147). When a change, bias, or drift is captured through monitoring, it often results in adjusting the dataset for model training and/or model optimization (PS190). Captured model performance degrades and invokes model re-training (PS114) or relevant ML processes (PS140). Other common monitored subjects are **resource usage**, e.g., memory usage and number of API calls (PS24), non-functional requirements like **security** (PS32). Monitoring changes around system dependencies helps providing notifications regarding specific modifications (PS109). Information collected during monitoring is stored and used to produce **notifications and reports** of important situations which often leads to an invocation of earlier tasks in the pipeline (PS151). Dashboards (PS114) and reports (PS102) are common methods for presenting this information. Moreover, monitoring enhances users' and business stakeholders' trust by enabling system observability (PS145), e.g., providing insights on how pipeline tasks are executed (PS91). While MLOps engineers are primarily responsible for monitoring (PS136), (PS111), other roles, such as product performance experts, site reliability engineers, and solution engineers, also utilize monitoring data to analyze issues (PS31). In more mature systems and organizations, such as those in highly regulated industries like financial services (PS99), monitoring also includes explainability features to enhance understanding of model predictions, including decision visualizations (PS17) and rule-based root-cause analysis (PS23).

3.2.6 Pipeline Management. An MLOps pipeline consists of interconnected ML life-cycle tasks and operations, which can become easily complex due to their dependencies. Pipeline management addresses pipeline design, orchestration, and optimization to execute MLOps tasks efficiently and reliably. **Pipeline automation** minimizes human intervention by executing MLOps tasks automatically, simplifying repetition of complex ML operations (PS42). Studies have explored automation for different use cases, such as pipelines for data downloading, processing, model training, and testing (PS3), or for model configuration, serving, and monitoring (PS44). Automation relies on **pipeline triggers** to enable task execution. Triggers can be sequential, i.e., executing a task upon the previous task's completion (PS42), periodical e.g., executing according to a pre-scheduled period (PS25), event-driven, e.g., deployment can be triggered when a new ML model is available (PS147), e.g., training can be triggered when new data is available or the model performance is reduced (PS140), or a concept drift is detected (PS102). Pipeline automation is often tailored to specific use cases. For instance, one study automates data downloading, processing, model training, and testing (PS3), while another focuses on automating model configuration, serving, and monitoring (PS44). Moreover, **scaling pipelines** is essential in today's infrastructures such as diverse environments including edge devices and multi-tenant platforms. For example, data pipelines must adapt to manage the evolving customer requirements (PS42), tools like Airflow support scalable, dependency-aware pipelines (PS152).

3.2.7 Artifact Management. Versioning artifacts is crucial in MLOps projects just as in traditional software engineering. Common MLOps artifacts are code, data, feature, models, experiments,

metadata, and project documents. Managing artifacts involves keeping a record of history and different versions of artifacts. It facilitates collaboration among team members (PS33), enables debugging and system backups (PS157), supports artifact reuse (PS32), ensures auditability for quality and risk assessments (PS24), (PS133), and enhances project transparency (PS41). Studies focus on versioning various MLOps components, including data science experiments (PS121), codes (PS24), ML models (PS65), training metadata such as dataset versions (PS133), parameter weights used for model training (PS157), tracking software versions running with the ML model (PS41), logs of pipeline executions (PS157), logs of monitoring events (PS33), and logs of container data (PS90). Further, data (PS24) and model (PS190) provenance are stored which are the records that keep the origin and lineage of data and model in ML enabled systems (PS66).

3.2.8 Quality Assurance. Quality assurance covers ensuring data quality, model quality, and ML application's quality. Quality is managed across various MLOps phases, but data quality is closely associated with data cleaning, analyzing data, model training, model selection, and monitoring phases (PS24). Model quality is managed through data collection, model training and evaluation, and monitoring phases (PS111). Moreover, ML application's quality is ensured through traditional software testing strategies (PS100). Data quality involves assessing the scientific quality of data with domain experts, e.g., whether data includes realistic values, correcting data anomalies, checking data completeness, e.g., addressing problem-related missing values, checking consistency in data format and structure, data heterogeneity, and fairness of collected data (PS62), (PS24), (PS151). Model quality involves assessing the model's predictive performance through performance metrics and test sets (PS21), validating model bias and fairness (PS100), maintaining ML portfolio health (PS90), monitoring model to detect overfitting issues (PS59), dimensionality reduction (PS24). Application quality involves unit testing, acceptance testing (PS89), maintaining containerized images and identifying vulnerabilities (PS90).

3.2.9 Team Management. Various people from different backgrounds, such as ML engineers and software engineers, need to work together in MLOps settings (PS150). Facilitating communication among these individuals is crucial to maintaining collaboration and ensuring smooth operation of ML system to achieve business goals (PS105). Activities associated with team management include clear identification of requirements, planning the project, ensuring transparency in project documentation and model evaluation, involving individuals with relevant expertise in meetings and negotiations, and investing in education of team members (PS150).

3.3 RQ1.3 - What Are the Practitioner Roles and Responsibilities in MLOps?

Domain experts possess deep knowledge of the problem domain, such as clinical neurologists for an ML-enabled psychological monitoring system (PS174) or urban planners for a smart city system (PS30). They are described as "irreplaceable actors within ML projects" (PS24) due to their crucial role in domain discovery (PS41), engineering domain specific requirements (PS24), and fulfilling business goals (PS41), (PS116). Moreover, they contribute to data collection, cleaning, labeling, hypothesis development, model evaluation, and result interpretation (PS32), (PS150), (PS111). They either provide domain knowledge and processed data to the MLOps team in the early stages of a project and/or collaborate directly with data engineers, data scientists and ML engineers during the ML phases to validate data and model (PS24), (PS24), (PS174), (PS173).

Business analysts define the problem, set clear objectives, and assess whether an ML-enabled system can provide a solution to the problem at hand (PS86), (PS175), (PS97). MLOps team collaborates with business analysts to gain a better understanding of the problem and convert it to a successful ML solution that aligns with the business goals. Their role may overlap with domain experts, as both contribute to understanding the problem domain. Business analysts also define

business requirements, **key performance indicators (KPIs)**, and model quality metrics, leveraging query processing on structured datasets to analyze business problems (PS86), (PS154).

Data engineers are mainly responsible for fundamental data activities, enabling data operations in a high-quality, secure, and scalable way (PS1), (PS24). Data engineers' typical set of tasks include identifying suitable data sources, verifying data legality, collecting data from various sources, cleaning them, and preparing labeled data (PS5), (PS51), (PS93). This data engineering process is characterized as "extract-transfer-load" which involves transferring data into consumable formats for data scientists and ML engineers (PS112), (PS106). Beyond data preparation, data engineers set up and manage data storage infrastructures, build automated data pipelines (PS1), and enable big data storage solutions such as centralized data lakes (PS69) and streaming platforms (PS124). Their role extends to security, privacy, quality, and governance of data (PS86), (PS106). Some organizations define specialized roles, such as **data curator**, responsible for custom data acquisition (PS19), and **data expert**, applying noise reduction techniques (PS5).

Data scientists are mainly responsible for exploring and analyzing the data made available by data engineers, training, tuning, and evaluating models to derive insights (PS121), (PS24). Particularly, their initial task is to understand the business problem and its domain, evaluate the feasibility of an ML-enabled solution (PS24), (PS114). At this step, data scientists work with domain experts and business analysts to discuss **objectives and key results (OKRs)** (PS86). Data scientists also conduct feature engineering to identify the best combinations of features for training effective ML models (PS104). Tasks such as preparing training, test, and validation sets, choosing suitable ML algorithms and training approaches, and selecting appropriate evaluation metrics (e.g., accuracy) are also carried out by data scientists (PS112). Moreover, as data engineers build the initial part of the ML pipeline, data scientists are responsible for designing and building scalable and high quality data analytics pipelines for experimentation and model training (PS114). Additionally, they ensure the trackability of multiple versions of experiments, datasets, and models (PS121). Also, they monitor the early results after the deployment, collaborating with business analysts and ML engineers, whether the model meets technical, business, and regulatory requirements (PS114). If necessary, data scientists iterate on the model to improve its performance (PS93).

ML engineer and data scientist roles often overlap, with some studies suggesting they are interchangeable (PS134), (PS140). However, many studies distinguish the role of the ML engineers with their primary tasks being model optimization, deployment, and monitoring (PS114), (PS154). Whereas, experimenting with datasets and models, and optimizing the model and its parameters are tasks shared with data scientists (PS117). In particular, ML engineers contribute to monitoring and production activities, collaborating with data scientists and business stakeholders to ensure the deployed model meets performance and business requirements (PS114), (PS107). They are also responsible for executing appropriate deployments, such as online serving, and monitoring the model's performance in the production environment (PS114), (PS107).

MLOps engineers are responsible for building automated pipelines, maintaining operational stability of the entire ML system, and facilitating collaboration across roles (PS106), (PS116). While data engineers, data scientists, and ML engineers design and implement the pipelines for data collection, analysis, model development, and deployment, MLOps engineers focus on building, optimizing, and integrating these pipelines, including services like cloud and containers (PS106). They manage dependencies, versioning, tracking mechanisms, and platform governance. MLOps engineers also monitor data, models, and applications along with ML engineers (PS116), and step in when human intervention is needed-such as when re-training is required due to data shifts or resolving dependencies between data and models, (PS136). Effective communication and collaboration management across roles during these tasks is also the responsibility of MLOps engineers (PS105).

DevOps engineers also play a role in ensuring the seamless deployment, integration, and stability of ML models within production systems. Their responsibilities often overlap with those of MLOps engineers, operational engineers, and software engineers (PS55), (PS69). While some studies indicate DevOps engineers in traditional software projects are replaced with MLOps engineers for ML projects (PS109), a few mention DevOps and MLOps engineers work within the same team. For example, Neghawi et al. (PS116) report that while software engineers develop and validate applications, DevOps engineers construct, test, and deploy working systems, and MLOps engineers continuously monitor the entire system to ensure operational stability.

Software engineers develop the final applications and services that support ML production. Application developer is another common reference to the same role (PS1). They often collaborate with data scientists and ML engineers in data processing and analysis, and model building (PS4), (PS150). Besides, deployment, packing, releasing, configuring, and monitoring of ML systems are also reported as the role of software engineers (PS161), (PS24), (PS154). Additionally, **quality assurance engineers** are responsible for testing the software application and services to ensure product quality (PS36).

Operational engineers or operational team are not consistently referenced across all studies. But some highlight their role as taking over after data scientists and/or ML engineers hand over the ML model (PS24). They are responsible for deployment operations including packaging, releasing, and configuring ML systems (PS24) and continuous monitoring of the model's performance (PS104). They serve as a bridge between data science and software engineering teams, facilitating iterative improvements by reporting issues, providing feedback, and ensuring the model remains efficient and reliable in real-world applications (PS67). In the absence of explicit mention of operational engineers, these tasks are carried out by ML (PS114), and sometimes software (PS24), and MLOps engineers (PS106).

Technical and solution architects are responsible for designing and building ML systems and services that serve as the backbone of ML operations in production (PS70). Their role includes secure design, selecting appropriate technologies and design paradigms, tracking and monitoring models in production, and integrating cloud-based applications and web services (PS105), (PS155). Also, referred as IT team or IT architects in the literature (PS105).

Security experts ensure the integrity, confidentiality, and availability of ML assets by defining security objectives and requirements at the project's initiation (PS105). Some studies refer to them as a dedicated security team (PS108) or security engineers (PS29), while others define them as data and software engineers with security expertise (PS105). Their role involves securing the overall ML system, addressing potential attacks during data ingestion and feature engineering, and enforcing secure coding practices (PS105).

Compliance teams focus on mitigating unfairness and bias in ML systems throughout the model training, deployment, and monitoring phases (PS108), (PS151). Recently, **risk professionals** have become involved in MLOps projects to manage data and model-related risks (PS155). Specifically, **model risk manager** is responsible for ML model inventory management, model explainability, and pipeline management (PS111), (PS106), while other studies only mention risk professionals for managing any data and technology related risks (PS155). **Data steward** and **data quality board** specifically focus on data quality management (PS128), and **data governance officer** focus on data governance and privacy (PS114).

Business owners, business managers, and business stakeholders are not clearly distinguished in the literature yet. However, they are often associated with organization level decision making (PS24), supervision of other roles in development and operations (PS105), and evaluating the production of ML systems from an operational and regulatory perspective (PS69). **Product experts, owners, and managers** are associated with defining and optimizing product features to enhance

performance and usability. For example, product experts identify feature set candidates based on specific performance issues and apply feature engineering techniques alongside to improve model outcomes (PS31). Product managers make data-driven decisions by reviewing relevant collected and visualized information (PS34).

Collaborative activities among roles: In a typical MLOps pipeline (Figure 3), various roles collaborate to fulfill different responsibilities. Often, one role is the primary owner of an activity, while other roles provide support as collaborators (PS86). Data engineers, data scientists, and ML engineers often collaborate with domain experts and business analysts during data collection and preparation, data analysis, and model development and deployment stages (PS106). Particularly, data engineers work together with data scientists and ML engineers to determine the appropriate data format for subsequent steps (PS111). Additionally, data engineers, scientists, and ML engineers often collaborate with business and domain experts to interpret both the data and the model (PS111). ML engineers and occasionally data scientists accompany the MLOps engineers at the initial monitoring to ensure the model's stability and step in when the model or features need optimization (PS116). Meanwhile, MLOps engineers interact with every role to ensure the optimization of the automated pipeline, the operational stability of the ML system, and the smooth collaboration across various roles (PS105),(PS3).

MLOps is perceived as a multidisciplinary field integrating continuous engineering practices (CI/CD), DevOps principles, data science, ML, and software engineering to systematically manage the lifecycle, ML-enabled software systems. It encompasses traditional ML activities with continuous deployment and monitoring, while placing increasing emphasis on *quality assurance and compliance* and *efficient management of pipelines and artifacts*, and cross-role collaborations. Emerging roles such as *MLOps engineers*, *data governance officers*, and *compliance teams* illustrate increasing specialization within MLOps, emphasizing its multidisciplinary character. Findings of RQ1 reflect the evolution of MLOps into a more structured, mature, and systematically governed approach for operationalizing ML systems.

4 RQ2: What Are the State-of-the-Art Practices and Techniques Proposed for Implementing MLOps?

Here, we report the state-of-the-art work practices, processes, techniques, and supporting technologies that aid organizations in implementing MLOps. These practices cover a broad spectrum of social and technical aspects across the life cycle of ML models. We have grouped and reported them under seven categories below. Moreover, we illustrate the practices and techniques by mapping them to the corresponding productionalization challenges in Figure 4.

4.1 Adapting Team Structure and Work Practices for MLOps

The productionalization of ML models is a multifaceted challenge that requires the expertise of stakeholders with diverse skill sets, from domain knowledge to data science, software engineering, system operation, and cyber security. Following best practices are introduced to enable the successful implementation of MLOps by fostering effective collaboration among such diverse stakeholders.

Establishing balanced project teams: The AWS Well-Architected ML framework (PS111) and Azure ML best practices (PS153) recommend establishing *project teams*, which comprise of domain expert, data engineers, data scientists, ML engineers, MLOps engineers, IT auditor, and cloud security engineers to work with ML-based software features and systems, rather than building functional silos for each capability, e.g. ML engineering team, security team. A project team should be configured with balanced capabilities such that it can carry out all phases of the life cycle of an ML model.



Fig. 4. MLOps practices and techniques, and the corresponding ML productionalization challenges.

Establishing specialized ML engineering teams as ML projects grow: As organizations grow in ML utilization, the number of ML projects maintained by a project team could rapidly outgrow the ability of the team to scale. At this state, a helpful practice is establishing a specialized ML engineering team that focuses on the deployment, operations, and maintenance of ML models. These capabilities are common across ML projects and uncoupled to domain specific ML problems. By centralizing these capabilities into a specialized team, an organization can develop, manage, and utilize these capabilities more efficiently. Specialized ML engineering team along with standardized processes and reusable patterns and artifacts lay a foundation for establishing “AI Factory”, scaling ML productionalization to ML *industrialization* (PS153).

Adopting a Git-based workflow: ML project teams can leverage Git-based workflows to coordinate and carry out the day-to-day activities of ML projects. Git allows team members to work on isolated branches without disrupting the stable version of the project as well as enabling the automation of CI/CD activities. Gunny et al. (PS131) propose a branch structure consisting of two stable branches (i.e., dev and main) and several short-lived feature branches to integrate model training, testing, and deployment activities into pull requests under two main stages. Pull requests from feature branches to dev branch would trigger model training and testing within a staging environment, thus ensuring the model’s performance. Pull requests from dev to main branch would trigger the packaging of the model as software containers and deployment of the packaged model into a production environment.

Integrate practices for cross-silo concerns across ML life cycle: Cross-silo concerns include AI fairness, AI explainability, and security. Instead of addressing these concerns at any particular phase, they should be a cross-silo effort, which each part of the life cycle implementing different

aspects. Amazon AI Fairness and Explainability Whitepaper (PS151) recommends integrating bias measurement before-, during-, and after-training, as well as in production. The detected bias then can be addressed using pre-processing, training tuning, and post-processing. Security is another cross-silo concern of ML life cycle. Zhang and Jaskolka (PS105) introduce the concept of **Secure Machine Learning Operations (SecMLOps)** based on the **People, Processes, Technology, Governance, and Compliance (PPTGC)** framework. SecMLOps embeds security controls across the whole life cycle. In the requirement phase, security objectives and requirements of ML models and data are defined, and security models are established. These models drive the development of security policies regarding data, model, physical security, personnel security, administrative security, and network security across the ML life cycle and all involved parties. These models also drive the development and integration of security monitoring and incident response components that monitor ML models in production.

4.2 Improve Transparency and Provenance Across ML Life Cycle

The goal of improving the transparency across the ML life cycle is twofold. Firstly, improving transparency helps project teams avoid or reduce duplicated work, such as building and running data pre-processing and feature engineering pipelines. A closely related advantage is avoiding unexpected dependencies between projects, such as via a shared feature set. Secondly, maintaining the provenance of data, models, and features facilitates project teams to identify the root causes of production issues and roll back ML systems to the previous working state when production issues happen. The improvement of transparency and provenance can happen along three axes: models and experiments, datasets, and features.

Applying version control to experiment codes and artifacts: Existing guidelines (PS153), (PS111) advocated using version control systems such as Git to manage all inputs that go into experiments, such as software codes for model training and testing, datasets, environment configurations, and other experiment parameters. Reproducibility is a key benefit of version control. However, the benefit of this practice extends beyond reproducibility. It provides project teams with the ability to roll back the system to the last known functional version of an ML system, thus facilitating recoverability and improving the overall fault tolerance of these systems.

Maintaining history of experiment runs: The probabilistic nature of many ML models makes it possible for a set of version controlled source codes and artifacts to produce ML models with varying performance characteristics across experiment runs. Therefore, it is beneficial to maintain the historical data of experiment runs of the model (PS153), in addition to the version controlled source codes and artifacts. Experiment versioning plays an important role in ensuring the repeatability of experiments and improving their conclusiveness (PS131). The historical data of experiment runs could provide additional transparency into the decision to upgrade models. This data would also be invaluable for planning the resource provisioning and optimization for the model training process. Experiment management tools such as MLFlow (PS164) and IBM AI Factsheets (PS165) enable experiment tracking along with model and data versioning (PS154). By integrating tools like AutoML with Experiment management tools, practitioners can automate the generation of the best suited models (PS154).

Tracking and Maintaining Lineage and Provenance of ML Models: Lineage and dependencies among models and datasets are other aspects that project teams need to capture to develop a comprehensive picture of ML model life cycle. Closely related to lineage is the model's provenance, containing its machine-readable metadata and historical events within its life cycle, such as how and by whom it was trained and tested. The IBM MLOps handbook (PS165) highlights two main needs for capturing metadata and recording historical events regarding the life cycle

of ML models: improving the productivity of ML model productionization by making ML model lifecycle related metadata visible to all stakeholders, and facilitating the regulatory compliance checking and explainability of the models. Academic literature proposes multiple provenance extraction tools such as Vamsa (PS129), Geyser (PS129), and MLflow2PROV (PS164). Vamsa is a knowledge-base driven *static* provenance extraction tool. Geyser extends Vamsa for dynamic provenance extraction, provenance storage and querying. MLflow2PROV was designed for extracting provenance from version control system, experiment management systems and store in a central database for querying and visualization. IBM AI Factsheets (PS165) capture model and process metadata such as model details, training information, metrics, and so on, and store in a central meta-store.

Defining concrete model management strategy: A model management strategy defines how models are identified, named, and labeled. An organization might also include requirements on the type and format of metadata, provenance, and lineage information to be maintained by project teams. A critical element of a model management strategy is the process and criteria for replacing models. For example, the Azure ML best practices (PS153) recommended labeling a baseline model as “champion” and new models as “challengers.” Strategies can be defined to leverage experiment results captured in the historical data of experiment runs to decide when and how to replace the “champion” model.

Establishing centralized data repositories: Centralized repositories provide project teams with visibility into the available data and its utilization to avoid redundant effort and potential dependency problems (PS155). Centralization of data also helps organizations to enforce a consistent access control policy and allows them to implement best practices regarding data privacy and bias that are relevant across projects. Centralizing data also facilitates the implementation of techniques such as real-time monitoring and alerting of data drifts (PS106) and automated detection of data dependencies. For instance, Boue et al.(PS145) propose a dependency mapping framework to analyze and notify MLOps engineers of changes in data sources and their effects. Another advantage of centralized data repositories is resource utilization efficiency because they allow organizations to employ specialized and scalable data technologies, such as transactional databases supporting structured or semi-structured data (PS133), data lakes (PS110), and data warehouses (PS154) where version controlled datasets are stored to maintain links between data, code, and models.

Establishing feature stores: Features can become a point of friction between the data science and operation sides of an ML model, especially if the feature construction code is not well documented and lacks scalability. In these cases, the feature engineering code might need to be reconstructed. However, this process might introduce subtle changes to the features that make model’s performance deviate from the experiment results. At the same time, reusing features and feature engineering pipelines might lead to undocumented coupling between projects and unexpected consequences of changes. Establishing centralized feature stores lays a foundation to address these challenges, because they provide visibility into existing features and feature engineering pipelines to avoid redundant effort. Moreover, feature stores facilitate feature governance by making features visible and reusable among different stakeholders in a secure manner (PS112), (PS154). Some existing feature store platforms and technologies include Amazon SageMaker Feature Store³ and Databricks Feature Store.⁴

³<https://aws.amazon.com/sagemaker/feature-store/>

⁴<https://www.databricks.com/resources/demos/tutorials/data-science-and-ai/feature-store-and-online-inference>

4.3 Continuous Monitoring Across ML Life Cycle

Establishing continuous model performance monitoring: The concept of “model performance” has multiple facets, such as prediction accuracy, robustness, fairness, explainability, and safety. Once the required performance metrics have been established during project initiation, they can be implemented as automated monitoring jobs. Various tools exist for monitoring production ML models. For instance, AWS Sagemaker Model Monitor (PS107), (PS166) enables detecting data drift, concept drift, bias drift, and feature attribution drift of deployed ML models. Sagemaker Clarify (PS151) can detect bias to ensure fairness of models deployed on Amazon Sagemaker. Similarly IBM OpenScale (PS165) monitors models for model accuracy, fairness and data drift. Databricks Lakehouse Monitoring (PS114) maintains inference tables with request-response data to detect data and model drift. Jayalath and Ramaswamy (PS144) highlight the importance of user feedback for the success of deployed ML models and propose a monitoring framework to that utilizes user feedback to investigate data deficiencies. Visual analytic tools such as Slice Teller (PS136) would be invaluable in aiding human experts in such cases when human inspection of model’s *local* performance on some particular segments of data is necessary.

Establishing continuous data monitoring and validation: As organizations have more data over time, they often face issues of data drift, bias, data quality and security (e.g., poisoning). Continuous monitoring and validation of datasets are practices to address these challenges. Data validation tools such as Tensorflow DataValidation⁵ and Amazon Deequ⁶ are used to write declarative data quality constraints to catch data quality issues as they occur. Further, Tu et al. (PS130) extend this concept to auto-program data quality constraints by leveraging statistical information from past executions.

Leveraging events to detect and response to model and data issues: The continuous monitoring of models and datasets can produce the signals to facilitate further analysis and response. Event-driven programming approach is used to take advantage of these signals (PS153). In particular, monitoring platforms are considered as event emitters. These events can be published on a centralized event bus that is available to all teams. Authorized team members can subscribe to events from the bus to be notified for further analysis and response.

4.4 Systemizing the Provisioning and Configuration of Infrastructure and Environment

The performance of ML models, particularly deep learning variants, depends not only on the input data and model parameters but also on the software environment surrounding them. Therefore, it is critical to maintain the consistency of the software dependencies and configurations across training, testing, staging, and production environments. Practices such as IaC, automated infrastructure provisioning, and Inference-as-a-Service provide the necessary systematization to address the environment consistency challenge.

Leveraging IaC to ensure consistency between environments: The IaC practice ensures that software environments are constructed and configured consistently according to predefined specifications. These specifications can be captured as software artifacts, stored in source code repositories, and managed by version control systems. Treating environment specifications as code artifacts enables project teams to collaborate on the development, maintenance, and improvement of these configurations. Existing IaC tools such as Ansible and Terraform can be readily adapted to

⁵https://www.tensorflow.org/tfx/tutorials/data_validation/tfdv_basic

⁶<https://aws.amazon.com/blogs/big-data/test-data-quality-at-scale-with-deequ/>

ML projects. Existing literature has also proposed ML-specific tools, such as the Pinto utility (PS131) that automatically creates software environment based on the specification of an ML project.

Leveraging Inference-as-a-Service: Inference-as-a-Service is an approach to centralize the execution of ML models to a service provider. Instead of setting up local environment to run a model, entities and software systems needing model's inference would request the inference results from the service provider via a predefined service API instead. By removing the need to run models locally, Inference-as-a-Service can reduce the number of production environments to be setup, thus mitigating the risk of organizations using inconsistent environments. Another advantage of this approach is the maximization of resource utilization: the centralization of AI inference can replace multiple under-utilized infrastructure across entities that run the model locally with a fully-saturated infrastructure at the centralized service provider (PS131). NVIDIA Triton Inference Server is an open-source software that could be leveraged to implement AI inference services.

Automate provisioning of training infrastructure: An advantage of the systematization of environment creation is the possibility to provide data scientists with the ability to provision training infrastructures such as virtual machines by themselves. For instance, AWS (PS111) provides managed training infrastructures for optimum automated resource provisioning during iterative training process through Amazon Sagemaker. Sagemaker monitors the resource utilization of GPUs, CPUs and network bandwidth to make auto scaling of training infrastructure, thus abstracting the infrastructure management activities from the users and reducing training costs through use of on-demand computing resources. Raffin et al. (PS133) propose the use of IaC and declarative configurations (i.e., Yaml manifests of Kubernetes defining the desired state of operation) to automate provisioning of underlying infrastructure in a reliable, reproducible, and scalable manner.

A related problem is the monitoring of system metrics (GPU, CPU, and I/O utilization) to provide insights into overall performance of the training infrastructure. Rauschmayr et al. (PS167) introduce a profiling tool for Deep Learning model training clusters, which can be integrated to Amazon Sagemaker as an add-on. Profile monitors the system utilization metrics related to training jobs to identify performance bottlenecks (GPU, CPU, and I/O bottlenecks) and provide insights for optimizing resource usage during model training phase.

4.5 Systemizing Model Deployment

The process of translating data science codes into production codes and maintaining them brings about a variety of challenges. Some patterns for model deployment and maintenance have emerged from the literature and best practices.

Using software container to package and deploy ML code: A common practice is packaging ML code with all of its dependencies within software containers, and then applying optimizations such as parallelization and load balancing upon the containerized workloads (PS179), (PS172). Containerized workloads can also be operated at scale by software container orchestrate platforms such as Kubernetes (PS140). By encapsulating ML models as docker containers practitioners can reap the benefits of container management platforms such as AWS Fargate which provides serverless options for ML model deployment (PS111). Kubernetes enables automated deployments, monitoring, and scaling of ML models to ensure efficient operation of deployed ML models, thus overcoming limitations of manual model deployments (PS127). Syrigos et al. (PS127) propose an extension for Kubernetes Scheduler where container scheduling decisions are made in a latency-sensitive manner in addition to CPU and RAM of the computing resources to deploy ML models.

Abstracting data acquisition: Deployed ML systems do not exist in vacuum. Instead, they need to pull data from various sources and perform pre-processing tasks such as merging datasets and

cleaning. Therefore, containerizing ML codes by itself is inadequate for addressing productionalization challenges. An approach to abstracting the data acquisition activities as services can help attain scalability of data processing with increasing volumes of data. (PS179) proposes a pattern to abstract data sources and introduce a software agent that pulls and feeds data to containerized ML code. This design introduces the necessary decoupling to parallelize the containerized ML code.

Applying patterns for upgrading models: To prevent potential failures after updates to ML models, various deployment strategies for a safe upgrade of ML models in production are proposed. AWS (PS111) highlights the importance of conducting a tradeoff analysis to compare multi-version deployment strategies like A/B testing, canary deployments, and blue/green deployment to select the best approach. Such strategies facilitate rollback or roll-forward decisions to be made after evaluation of the performance of the updated model without affecting the availability of the deployed service. To this end, Guissouma et al. (PS168) propose a multi-version execution platform that facilitates the execution of shadow versions alongside the current active version, measures safety metrics of the updates through monitoring, and safe activation of the updated version if it outperforms the current active version.

4.6 Setting up Automated Pipelines for Cross-silo Activities

Developing and utilizing automated pipelines: Teams might be able to save time by having their well-defined model validation pipelines approved, saving time on having the models themselves validated and approved. MLOps pipelines can be implemented with various tools. At a basic level, software scripts (e.g., Bash or Python) for submitting training or testing jobs can form a rudimentary version of automated pipelines. Moving up a level, organizations can adapt the existing tools and platform for CI/CD to MLOps workflows. GitHub Actions, CircleCI, ArgoCD, and Jenkins were among the tools suggested by the existing literature and industry best practices for adaptation to MLOps. For instance, Stieler and Bauer (PS135) proposed a Git workflow for an “active learning life cycle”, which contains four interconnected iterations of data management, model training, software development, and system operations. In the proposed workflow, a CI/CD tool hosted on a resource-rich machine with GPUs executes ML pipelines for training, testing, and integrating ML models based on the code committed by developers to a source control management system. Alternative, generic workflow engines such as Apache Airflow⁷ can be adopted to MLOps pipelines. For instance, the FLScalize (PS172), an MLOps platform for federated learning, utilizes Airflow to orchestrate the interaction sequences among federated learning servers and clients as well as other monitoring workflows. Finally, there have been more ML-specific workflow orchestration tools (e.g., SageMaker Pipelines, IBM Watson Pipelines, Google Vertex AI Pipelines, Netflix Metaflow, ZenML) for defining and running MLOps pipelines.

Optimising and automating MLOps pipeline creation and management: By specifying pipelines explicitly, it is possible to automate, optimize, and manage the execution of MLOps pipelines. The automation helps overcome the uncertainties that ML workflows face such as changing requirements, changing system context due to other workflows competing for resources, workflow orchestrator’s failure, or hardware crashes (PS170). For instance, Scanflow-K8s (PS170) was proposed as a framework for autonomic management and supervision of ML workflows in Kubernetes clusters. Moreover, automating the creation of MLOps workflows contributed to reducing the setup overhead, particularly during the initial states of ML development. Carqueja et al. (PS128) proposed a lightweight **Quick Machine Learning framework, QML**, to address this problem. Automated creation also facilitates optimized deployment of data analytic pipelines

⁷<https://airflow.apache.org/use-cases/mlops/>

against the heterogeneous infrastructure available for training or operating the models. Díaz-de-Arcaya et al. (PS169) proposed Orfeon, an AIOps framework for optimizing the deployment of analytical pipelines, taking as input the static pipeline definition by data scientists and the dynamic metrics from the target infrastructure.

Simplifying the integration of software tools and components into MLOps pipelines. Software-centric integration approach (PS171) is proposed in Looper where predefined API end points are provided for interconnecting steps in MLOps pipelines. Modularization through microservice-based design is recommended in (PS111), (PS119), (PS126) to enhance the integration of pipeline components.

4.7 Abstracting MLOps Complexity

Integrated development platform for ML Systems. Integrated development platforms provide a collaborative environment for whole-life-cycle-management for ML models and abstract the complexity from end users (e.g., data scientists, researchers, and other ML practitioners) (PS171). They provide interactive control planes accessible by ML practitioners to manage end-to-end ML life cycle (PS154). Managed cloud-based platforms such as Amazon SageMaker (PS108) and IBM Watson (PS4) provide support for data preparation, model training and experiment tracking, automated deployment on managed infrastructure and monitoring throughout the life cycle. Sagemaker (PS108) is developed with a modular and customizable architecture to support the integration of open-source tools to meet practitioner requirements. MLflow, Kubeflow, and Metaflow are open-source end-to-end platforms to simplify the ML lifecycle (PS109). Databricks Lakehouse Platform (PS110) provides a data-native platform which together with MLflow provides a single platform for managing data, conducting data analysis, model training, and model serving. Peng et al. (PS196) propose a platform for deploying models within Kubernetes and Istio to enable cloud-native environments by providing Source to Image (S2I) and model deployment support.

Reference Architecture of an MLOps system. Functional decomposition and reference architecture of MLOps systems guide the composition or development of MLOps pipelines and platforms depending on domain-specific requirements. To this end, academic literature presents domain specific architectures for various domains such as manufacturing domain (PS133), Industry 4.0 (PS126), transnational ML in health domain (PS132), and Gravitational Wave Physics (PS131).

State-of-the-art MLOps practices and techniques prioritize automation and optimization of MLOps lifecycle, service-oriented deployment, artifact tracking, and collaboration. Recent practices increasingly focus on domain-specific reference architectures tailored for particular contexts, systematic artifact management emphasizing provenance and lineage tracking, and frameworks and practices that support addressing compliance, fairness, and security. These developments illustrate an evolution toward more structured, scalable, and comprehensive MLOps implementations, capable of addressing emerging complexities in ML operationalization across diverse application domains.

5 RQ3: What Are the Challenges Practitioners Face when Adopting MLOps, and What Solutions have been Proposed to Address Them?

In RQ2 we discuss work practices, techniques, and technologies that organizations need to implement to enable MLOps. However, there are challenges that hinder teams from efficiently adopting MLOps into their organization's ML-based software system productionalization process. Thus, we identify such challenges and highlight reported solutions from the literature, and summarize them.

5.1 Socio-Technical Challenges

MLOps is a continuously evolving, technology-intensive domain which requires a considerable cultural shift in organization for its successful adoption (as discussed in Section 4.1). Our thematic analysis of literature resulted in three major challenges arising due to interactions between social and technical aspects when adopting MLOps.

5.1.1 Cultural Changes. The introduction of MLOps practices disrupts the processes followed during the development and management of traditional software systems. Thus, the successful adoption of MLOps practices depends on the openness and willingness of the teams to change and embrace the cultural changes introduced by MLOps (PS103). Works such as (PS103), (PS104), and (PS108) discuss the inter-disciplinary collaboration requirements of the MLOps practices as one of the major cultural changes. Successful adoption of MLOps requires continuous collaboration among interdisciplinary teams that previously used to work in silos, which may result in conflicts. As a solution to this, it is important to have MLOps best practices in place to enable smooth cultural transitions. One way to achieve this is the introduction of a clear and well-structured communication plan, and having a clear definition of the business objectives/goals and metrics as possible solutions to help teams understand novel MLOps practices introduced to organization culture (PS103). Furthermore, methods need to be implemented for swiftly identifying and resolving conflicts arising related to responsibility boundaries and different goals/ expectations of multi-disciplinary stakeholders (PS104), (PS150). Sothilingam et al. (PS104) propose a i* Strategic Actor Rationale to model the actor relationships to identify and resolve conflicts during collaboration. However, addressing the challenges associated with cultural changes is unique to each organization. For example, in the financial services domain, due to the complex regulatory environment, teams have to operate in silos (PS108). Thus, financial service institutions benefit from tools and platforms (e.g., ValidMind⁸) that enable continuous collaboration between data scientists and compliance teams to achieve efficient risk management throughout the productionalization lifecycle. Hence, necessary cultural changes must be implemented (through MLOps best practices and tools) after careful analysis of the existing organizational structure and strategically implement the changes to maximize their positive impact (PS103). To this end, Nvidia (PS154) recommends a human-centric approach to guide the selection and adoption of MLOps tools, techniques, and practices.

5.1.2 MLOps Fragmentation. Rapid evolution and growth in MLOps practices, solutions, and tools create industry-wide fragmentation of MLOps due to the increase in **complexity** and **diversity** of MLOps. Being a broad domain consisting of practices, processes, solutions, and tools to productionalize ML systems, MLOps is inherently complex (PS154). Moreover, the popularity of MLOps and the use of ML in various domains that lack expertise in ML (e.g., physical sciences (PS131)) result in the introduction of diverse approaches and tools particular to different domains, use cases and organizations (PS154). This results in accidental complexities, and it is challenging to separate accidental complexity from essential complexity (PS154). Standardizing MLOps process, tooling, and so on and generation of domain-specific reference architectures to capture domain-specific ML productionalization requirements contribute to solving these issues. For instance, (PS133) extends generic MLOps to meet the requirements of the Manufacturing Domain where ML-enabled software is deployed within distributed Edge-Cloud environments. However, as production ML systems are novel and the rapid evolution of MLOps, standardization needs a huge effort and should be done by bringing practitioners, researchers, and industry MLOps solution providers together.

⁸<https://validmind.com/>

5.1.3 Responsible MLOps. Achieving Responsible MLOps is a challenging multidisciplinary effort that brings people, technology, and processes together throughout the MLOps life cycle (PS105). This includes various aspects such as compliance with regulations and standards, AI fairness assurance through bias detection/mitigation, transparency and accountability through explainable AI and security. However, the MLOps process lacks explicit consideration for responsible AI practices and provides only limited guidance concerning these aspects, which makes their implementation challenging for practitioners (PS105), (PS120). Being a fast-evolving field, there's a shortage of experts who can navigate these challenging aspects successfully to implement responsible MLOps (PS120). Moreover, varying regulatory requirements across different domains demand different levels of collaboration with compliance teams and quality assurance teams throughout the MLOps lifecycle. For instance, highly regulated domains such as financial services require compliance by design as an MLOps practice. Developing comprehensive MLOps frameworks by integrating responsible AI practices can help practitioners overcome this. As an initial step, academic literature presents conceptual frameworks that integrate security (SecMLOps framework presented in (PS105)) and explainable AI practices (Explainable AI framework presented in (PS116)) into the MLOps process.

To facilitate Responsible MLOps, practitioners require tools that support end-to-end assurance of these aspects, which includes provenance support tools during developments and experiment activities (PS114), tools for bias mitigation during different stages of the ML life cycle and explain models and their predictions (PS151), and tools for automated ML compliance checking (PS105). Moreover, detailed documentation regarding responsible MLOps support of the tools and MLOps platforms can enable users to make informed decisions (PS105) on which tools to use. As an example, (PS106) provides the architectural components of a secure MLOps platform using AWS services, which can be used by practitioners to decide if AWS services meet their responsible AI requirements.

5.2 MLOps Pipeline Related Challenges

MLOps best practices recommend using automated pipelines for managing the life cycle of the ML-enabled software systems (as discussed in Section 4.6). This section details challenges faced by practitioners during the implementation of MLOps pipelines.

5.2.1 Complexity. While there are many powerful tools such as Kubeflow, FedML, Polyaxon for creating MLOps pipelines, high infrastructure cost, high expertise required for pipelines operation, high setup and maintenance overhead of the pipelines slows down their adoptions (PS128). This is prevalent especially during the initial stages of ML projects or during ML projects driven by small companies. In order to avoid these complexities, practitioners tend to choose manual workflows over automated pipelines. IBM's 2022 Global AI adoption Index report [10] also highlights that the success rate of MLOps adoption of small companies is considerably lower compared to large corporations due to existing complexity. Design and development of MLOps pipeline creation frameworks with low setup, maintenance overhead, and low infrastructure cost can overcome these challenges. DagsHub⁹ and QML (PS128) are lightweight frameworks designed to act as an abstraction layer between ML practitioners and their chosen pipelines and enable creation of lightweight pipelines to meet ML workflows.

5.2.2 Optimized Deployment. IoT domains require MLOps pipelines to operate within distributed computing environments which include resource constrained and geographically distributed edge devices and centralized cloud data centers (PS126). This creates the challenge of optimally placing modularized pipeline components across these resources, especially to meet the increasing resource

⁹<https://dagshub.com/docs/>

requirements during model training and model serving phases (PS127). This challenge is further amplified by the availability of continuous dynamic data streams. Thus, manual provisioning of pipeline components is error prone and provides sub-optimal results. Strategic modularization of pipeline components to suit the business domain characteristics and designing dynamic resources provisioning frameworks and algorithms can help overcome these issues (PS126), (PS127). Furthermore, training pipelines need to adjust dynamically (i.e., resource optimized model training, model splitting, and so on (PS125)) to enable parallel or distributed model training to ensure efficient operation of the MLOps pipelines.

5.2.3 Managing Large Scale Pipelines. Large-scale enterprises that adopt MLOps practices have to operate thousands of recurring and interdependent production pipelines. Such pipelines are prone to silent failures caused by data quality issues in the data pipelines such as schema-drift, increasing nulls, change of units, change of value standards, and change of data volume (PS130), (PS176). Such issues generate cascading failures in pipelines. Thus, practitioners face the challenge of maintaining large-scale pipelines without failures. Moreover, due to the sheer scale of pipelines and their complexities, manual monitoring of data pipelines and manual root cause analysis for model mispredictions incur extra human cost (PS130) (PS177). To overcome these challenges, practitioners should automate pipeline operations monitoring as much as possible. This includes automation of data quality issue detection (PS130) and automated root causes analysis for detected errors (PS177).

5.2.4 Continual Learning. With the popularity of IoT, streaming big data is available for training powerful ML models to derive insights from data (PS124), (PS123). However, high volume and velocity of streaming big data make batch training infeasible due to cost of storage and cascading lag created by storing and retraining stages on recurring pipelines. While continual learning pipelines have emerged as a solution to this, they need to be scalable enough to withstand high-velocity data streams and make swift model updates accordingly. To enable executing model updates without pausing the inference pipelines, practitioners need to create and use elastically scalable online learning pipeline architectures with dynamic approaches to scale pipelines components vertically and horizontally (e.g., StreamMLOps (PS124)).

5.2.5 Automation Decision Dilemma. Although the practitioners tend to automate wherever possible when implementing MLOps, ML lifecycle consists of activities where human feedback is crucial and full automation can create negative impacts, especially considering the responsible AI practices (PS110). For example, model validation processes require communication and approval from regulatory agencies, fairness and bias identification and analysis, root cause analysis from monitoring alerts, and so on. Moreover, companies may have to prioritize other tasks over allocating time and effort on automating some of the tasks. Thus, caution must be taken during automation such that the tedious tasks are automated and human insights are incorporated wherever it is needed throughout the lifecycle (PS114), (PS110), (PS116).

5.3 MLOps Platform Related Challenges

MLOps platforms are designed for end-to-end ML lifecycle management, as a means of abstracting the complexity of implementing MLOps as discussed in 4.7. This section highlights challenges and limitations related to creating and using such platforms covering platform provider and practitioner perspectives.

5.3.1 Platform Customization. For companies looking to build and deploy ML models, the main goal is to generate business value from data. Thus, for most of the practitioners, the underlying MLOps platform is only instrumental and their aim is to minimize the effort on implementing and managing the platform. Cloud-based managed MLOps platforms like Sagemaker are advantageous

to meet this requirement. However, using fully managed MLOps platforms means the practitioners would lose fine-grained control over the technology stack and the services supported by the platform may not meet specialized requirements of many businesses (PS108). This drives practitioners to implement and manage their own platforms which add additional complexity and cost. Practitioner expectation in this scenario is easy customization of managed platforms by integrating open-source, proprietary or homegrown tools. To meet this requirement managed MLOps platform providers need to design and implement their platforms following a modular architecture, integration, and customization support to enable practitioners to integrate open-source tools. As an example, AWS highlights the design of Amazon SageMaker which provides practitioners the flexibility to integrate their own workflows with open-source APIs supporting TensorFlow, PyTorch, Kubeflow, and so on (PS108).

5.3.2 Scalability. Services provided by MLOps platforms need to maintain performance requirements as the amount of service requests fluctuates and MLOps platforms should facilitate practitioners to create scalable workflows. In such scenarios, the lack of scalability of the underlying platform design becomes a significant challenge. As an example, the use of monolithic architecture would considerably impact the scalability of a system. Thus, for MLOps platforms **Microservices Architecture (MSA)** demonstrated high scalability with independently scalable but interconnected services (PS119). It is important to highlight, scalability of MSA based MLOps platforms can be achieved by using dynamic pipeline deployment approaches (discussed in Section 4.6) to ensure the efficiency of MLOps pipelines to enable large-scale learning and continuous deployment of ML models.

5.3.3 Artifact Management. Compared to traditional software, types and amount of artifacts related to ML-enable software system is considerably higher. This includes software codes, ML models and related hyper parameters, datasets, and other metadata such as provenance data used for improving the trust of the ML-enabled system (PS24). This is further complicated by the highly explorative and iterative nature of the ML model building process where the experiments include different datasets, data preparation techniques, algorithms, hyper parameters, and so on. Thus, versioning, traceability, and reproducibility become challenging (PS164). Lack of support for conjugated management of these artifacts is a challenge faced by practitioners when using MLOps platforms (PS121). To overcome this, MLOps platforms need to be built with version control, storage, and governance (access control, authentication, and authorization) support for data, model and associated meta-information (PS121).

5.3.4 Distributed Computing Environments. Edge computing is introduced as a means of improving data privacy by processing data closer to the sources. This has created paradigms such as TinyML (i.e., ML models running on very resource constrained heterogeneous edge devices) and Federated Learning (i.e., a method of collaboratively training a shared model without transmitting data to a central location). These aspects affect certain ML lifecycle activities compared to ML model deployment in cloud. For example, in distributed edge computing scenarios, the model selection depends on the edge devices it is deployed on due to the heterogeneous nature of the devices, and model retraining needs to be carried out in decentralized manner using distributed streaming data. Thus, existing Cloud-based managed platforms do not meet the requirements for TinyML and Federated ML scenarios at the edge. Thus, platforms need to be designed and developed (TinyMLOps platform (PS122)) considering such special requirements arising in distributed computing environments (PS123), (PS124).

5.3.5 Migration to MLOps Platforms. As MLOps is a new and upcoming practice, many companies are migrating already existing projects to MLOps platforms to enable efficient production

of the ML components of their software systems. Moreover, in ML-enabled software systems ML components are a small part of larger traditional software systems where CI/CD is achieved through DevOps practices and tools. So these companies utilize DevOps tools used for traditional software development for initial management of ML projects. Thus, when migrating MLOps platform requires additional information (artifacts and their links, workflow for creating pipelines, underlying resource requirements) needs to be extracted, which is a time consuming task (PS178). It is important for practitioners to conduct the migration process strategically by carefully documenting ML components and artifacts available in the current project, understanding the current architecture of the project to determine the best suited platform and utilizing tools supporting CI/CD support for software systems containing both traditional software components and ML components (PS178).

Practitioners face significant challenges across socio-technical shifts and barriers, ML pipeline complexity and fragmentation, scalability issues, distributed computing environments, the need for context-specific adaptations, ensuring compliance, fairness, and security of ML systems, and decision dilemmas about migrating to MLOps platforms and balancing automation with human involvement. Proposed solutions to these challenges reflect a shift toward human-centric, systematic, and transparent communication strategies, emphasizing domain-specific architectures, enhanced governance, systematic artifact management, and structured human-in-the-loop interventions.

6 Future Research Directions

Based on the MLOps practices, adoption challenges and proposed solutions, we identify the future research directions to facilitate MLOps adoption.

Domain specific and expertise driven frameworks to facilitate cultural changes. Challenges such as cultural shifts, integration of responsible and secure AI practices, MLOps fragmentation, and transitioning from traditional DevOps technologies highlight the importance of sharing practitioner experience and knowledge to drive standardized MLOps adoption. To this end, future research can contribute to developing frameworks consisting of case studies to facilitate cultural transitions in a context-aware manner, and address best practices, solutions, and tools employed to meet domain specific requirements. These frameworks need to be rooted in knowledge and experience sharing and designed to evolve as MLOps adoption increases. Studies such as (PS120) and (PS178) attempt to organize practitioner experience, but do not provide appropriate frameworks that can sufficiently capture the said aspects.

Domain specific MLOps reference architectures. MLOps architectures, tools, and technologies developed for different domains, e.g., manufacturing domain, health domain capture unique requirements for ML system productionization. While academic literature such as (PS133), (PS126), (PS131), and (PS132) makes contributions toward this, the industry-driven and practitioner-oriented nature of MLOps demands reference architectures that combine practitioner input with platform and tool provider expertise. Future work on reference architectures should integrate important aspects like security, ethics, transparency, compliance and governance, and so on to enable responsible and secure AI throughout the ML life cycle.

Evaluation frameworks for MLOps platforms and tools. The current literature lacks comprehensive frameworks that enable evaluation of MLOps platforms and tools, hindering standardization and effective navigation of practitioners in this space. Academic works such as (PS105), (PS128), and (PS121) evaluate existing MLOps platforms based on criteria like secure ML productionalization support, pipeline setup complexity, and artifact management support. Industry white papers and case studies such as (PS110), (PS106), (PS151), and (PS108) highlight the platform capabilities in automation, secure pipeline creation, responsible AI support, collaboration, and architecture.

However, this knowledge is scattered and does not provide a structured and comprehensive method for practitioners to evaluate which platforms and tools suit their needs the most.

Automation impact analysis frameworks. Although the proposed best practice for automation dilemma is to automate tedious tasks and incorporate human feedback where possible (PS110), existing literature does not provide a structured approach to do this. Given that the automation decisions depend on contextual factors such as industry domain, business objectives, regulatory requirements, organization culture, availability of tools, and so on, impact analysis framework needs to be created to analyze the effect of automation in a context-aware manner. Such analysis framework would help practitioners assess benefits and drawbacks using quantifiable metrics representing different criteria such as efficiency, scalability, cost, cascading risks, and so on, thus enabling them to make informed decisions to avoid under or over automation.

Adoption of modular architectures such as Microservice Architecture. Adoption of Microservice Architecture for MLOps pipelines and platforms facilitates their efficient and scalable deployment across distributed edge and cloud environments to support novel paradigms such as TinyML and Federated ML. Academic research on this demonstrates the potential of adopting microservice architecture (PS119), (PS179). However, the success of this approach depends on using strategic decomposition techniques for determining microservice boundaries without introducing unnecessary complexities. Moreover, management of such modular and distributed architectures requires development and integration of dynamic deployment, scaling, orchestration, and load balancing techniques along with data and artifact management to ensure their secure availability across distributed microservices.

Efficient management techniques for MLOps pipelines and platforms. Efficient pipeline component management techniques play a vital role in navigating the complexities of modular architectures, large scale pipelines, and continual learning and delivery of ML models in Big Data environments. Future research should focus on online and dynamic resource provisioning, dynamic scaling, and fault tolerance enabled by distributed and scalable monitoring of pipelines. Moreover, these techniques need to consider the resource heterogeneity, and limited and distributed resource availability at the edge.

Secure MLOps pipelines and platforms. Practitioners can combine managed platforms with open-source, vendor supplied, or in-house tools to create pipelines. Security and trust of these components ultimately affect the security of the ML models produced. Security breaches can compromise data privacy, model integrity, overall system availability, and so on. Proactive and reactive mechanisms need to be developed to monitor these pipelines and platforms and mitigate security breaches. As MLOps pipelines are created by integrating tools from various vendors, software supply chain vulnerabilities also become a considerable security concern. Intelligent approaches are needed to ensure secure configuration using technologies such as IaC and **Policy as Code (PaC)**. Moreover, with the rise of Edge-AI applications, pipelines and platforms need to be secured within distributed computing environments with limited resources.

Open source tool integration support. To meet different business requirements, customization of MLOps platforms is required where practitioners attempt to integrate open-source tools with managed MLOps platforms (PS108). In this case, interoperability is achieved through standardization of interfaces, model and data formats, protocols, and so on. Moreover, integration of open-source tools can cause security risks, requiring analysis before integration and continuous audits to detect and mitigate any risks after integration (PS108), (PS105).

MLOps tools for responsible AI. As highlighted in both academic and gray literature such as (PS151), (PS105), (PS164), responsible AI tools (i.e., bias detection and mitigation, provenance

Table 3. Comparison of Related Work

Study	Is multivocal	MLOps Conceptualization	MLOps Activities	MLOps Roles & Responsibilities	Practices & Tools	Challenges	No. Primary Studies	Timeframe
[14]	No	○	●	●		○	27 A	Up to May 2021
[15]	No	○		○	○	○	30 A	Up to Jul. 2021
[13]	No				○		24 A	2017 - 2021
[8]	No	●	○		○		69 A	Jan, 2012 - May, 2022
[17]	No	●	○	○			18 A	2015 - 2022
[22]	Yes	○	●			○	79 A, 72 G	2010 - 2021
[11]	Yes		○		○		8 A, 15 G	Jan, 2015 - Mar 31, 2021
[16]	Yes				○		1 A, 8 G	Gray lit.: Up to Apr, 2020, Ac. lit.: Up to Feb, 2019
Our work	Yes	●	●	●	●	●	150 A, 48 G	2013 - Sep, 2023

●: Full coverage, ○: Partial coverage.

A: Academic literature study, G: gray literature study.

extraction, compliance checking) need improvement. Future work on responsible AI tools requires input from experts across different domains having various levels of security, fairness, regulatory, governance requirements that span across the ML life cycle.

Conjugated artifact management solutions. With the increased types and number of artifacts generated during MLOps productionization, conjugated artifact management solutions are required to track the artifacts while maintaining relationships among them (PS121). Future solutions should also focus on governance along with efficient and scalable storage of artifacts across distributed environments.

7 Related Literature Reviews

We examine existing systematic review studies related to MLOps and qualitatively compare them with our work based on the type of considered primary studies, the RQs covered, the number of primary studies included, and the time period for study selection. We identified eight related studies: three MLRs, which collected data from both academic and gray literature, and five SLRs, which relied solely on academic literature. Table 3 presents a comparison of these reviews with our work, highlighting our contributions.

Kreuzberger et al. [14] reported a mixed-method research, where they conducted an interview with practitioners in addition to an SLR. Their work targeted one RQ: “what is MLOps”, which they answered with an overview of MLOps activities, roles and responsibilities and used these insights to create a unifying definition for MLOps. While they also discussed several open challenges related to adopting MLOps, it is not the main objective of the study.

Lima et al. [15] reported an SLR, where their RQs aimed to identify activities related to ML model deployment, MLOps related roles and responsibilities, operationalization tools for ML models, and challenges in ML model deployment. However, this study lacks comprehensive data synthesis to provide indepth analysis of roles and responsibilities. Additionally, their discussion on MLOps related technologies is limited to MLOps platforms extracted from academic literature and their discussion on challenges does not elaborate on the challenges related to MLOps adoption.

Kolltveit et al. [13] reported an SLR on the operationalization of ML systems, which covers all steps following model training and evaluation. Specifically, they answered four RQs focused on tooling and infrastructure related aspects of MLOps such as; how are ML models operationalized, what are the main challenges in operationalizing ML models, what tools and software infrastructure are used to operationalize ML models, what are the feature gaps in the tooling and infrastructure used to operationalize ML models. However, since the focus is only on tooling and infrastructure for operationalizing ML models, this work does not provide a comprehensive view of MLOps best practices considering the entire ML lifecycle.

Faubel et al. [8] reported an SLR on the technical overview of MLOps. Specifically, the study presented MLOps definitions and listed the most common activities, and listed tools for automating activities in MLOps. However, this review does not consider roles and task distributions in MLOps, thus lacking emphasis on socio-technical aspects in MLOps.

Mboweni et al. [17] reported an SLR on the state-of-the-art DevOps practices in ML-enabled systems. Specifically, this work aimed to report how MLOps is understood and differs from DevOps. The reported content provides a higher-level overview of MLOps definitions, presents key roles of MLOps and associated tasks, and describes MLOps activities by constructing an MLOps architecture covering ML lifecycle.

Steidl et al. [22] reported a mixed-method investigation (i.e., SLR and a semi-structured review study) focusing specifically on continuous development pipelines of AI models, which is one aspect of MLOps practices. This work focused on analyzing definitions of various terms related to continuous development pipelines for AI such as DevOps for AI, CI/CD for AI, MLOps, and so on, tasks handled by these pipelines, and the challenges related to implementation, adaption and usage of pipelines for continuous development of AI.

John et al. [11] delivered an MLR on the adoption of MLOps in practice. More specifically, the authors derived a framework for MLOps that shows the activities that take part in the continuous development of ML models and maps the practices of companies to MLOps maturity model to show how practices evolve through stages. However, they provided a high-level picture rather than elaborating on details regarding these questions.

Lwakatare et al. [16] presented an MLR on the RQ of “how to enable modern software development process with CD for AI-enabled systems”. The authors considered one academic and eight gray literature items. The study discusses tools and techniques used in the CD of ML systems. Important details are available concerning the development stages of ML-enabled systems.

Our analysis of the secondary studies presented in Table 3 demonstrates that no existing work contributes to building a holistic and timely depiction of MLOps landscape by answering all three RQs. Also, the majority of the related work relies solely on academic literature [8, 13–15, 17]. Given that MLOps is a rapidly evolving, practice-oriented field, insights from gray literature sources such as industry white papers and company websites offer valuable and timely perspectives on how MLOps is perceived by the community and adopted in real-life settings. Furthermore, the rapid improvements in MLOps call for a systematic review of the most recent literature to provide a timely view of the MLOps landscape. However, coverage by existing reviews extends only until 2022.

Considering these, our MLR aims to provide a comprehensive conceptualization of MLOps by integrating insights from a broader range of primary sources, from 150 academic studies with more practical knowledge from 48 gray literature items published up to September 2023. Our study extends prior research by linking MLOps conceptualization to productionalization challenges, identifying emerging activities, roles, practices, challenges, and solutions. Furthermore, we discuss future directions, emphasizing the need for domain-specific frameworks, secure pipelines, automation impact analysis, and responsible AI tools, aiming to enhance MLOps efficiency.

7.1 Evolution of MLOps

We share common RQs with related work. Hence, we compare our findings with those reported in prior studies to illustrate how MLOps has progressed over time.

Conceptualization: The conceptualization of MLOps in earlier systematic studies aligns closely with our RQ1.1 results (e.g., [14], [17]). This indicates that MLOps definitions in the literature have remained largely consistent from 2022 (the latest timeframe in prior studies) through September

2023 (our study's timeframe). Also, earlier studies explicitly defined MLOps to clarify its conceptual boundaries (e.g., (PS78)), whereas recent papers directly use the term without definitions, assuming community familiarity (e.g., (PS188), (PS24), (PS144)). This shift highlights the growing maturity and widespread acceptance of MLOps as a recognized research area.

Activities: While our findings align with previous works regarding core MLOps activities, we identify newer focal areas such as security (PS105), expanded quality assurance which includes data and model aspects (PS191), compliance and governance processes (PS106), and enhanced collaboration and communication management (PS150). The increased prominence of these aspects in recent literature signals a maturation in the understanding and structuring of MLOps practices (Section 3.2).

Roles: Recent literature emphasizes new roles focusing on quality assurance, compliance, and security concerns, such as data stewards, data quality boards, and compliance teams (PS106), (PS114). Although an earlier study identified the MLOps engineer role [14], our findings explicitly differentiate its responsibilities from those of ML engineers, positioning it within clearly defined workflows and clarifying its interactions with other emerging roles (Section 3.3).

Practices and techniques for implementing MLOps: Prior systematic studies identified practices like workflow automation, artifact tracking, collaboration, and continuous monitoring. Our findings extend these set of practices by highlighting recent developments, including the use of domain-specific reference architectures for specific business contexts (PS133), pipeline optimization strategies to enhance scalability and performance (PS135), containerization and service-oriented architectures (PS131), and systematization of regulated ML tasks to ensure compliance with emerging governance requirements (PS116) (Section 4).

MLOps adoption challenges and solutions: Challenges such as ML pipeline complexity, socio-technical barriers, scalability and performance issues, distributed computing environments, and adapting MLOps to specific contexts have been reported in related work. Beyond these, our study highlights more recent and nuanced issues such as compliance (PS111), fairness (PS151), and security of ML systems (PS105), customization of ML platforms (PS108), pipeline fragmentation due to tool diversity (Section 5.1.2), and decision-making challenges regarding automation and migration (Sections 5.2.5, 5.3.5). We observe that solutions to these challenges have correspondingly evolved, increasingly incorporating human-centric, transparent, and plan-focused approaches for cultural and cross-silo adaptations (PS103), domain-specific MLOps architectures, frameworks supporting compliance, fairness, and security, as well as human-in-the-loop interventions (PS110) (Section 5).

8 Conclusion

We have systematically selected and analyzed 150 relevant academic and 48 gray literature studies to provide a comprehensive overview of the current MLOps landscape by addressing three main RQs: (1) Conceptualization of MLOps, (2) What are the state-of-the-art best practices and techniques for implementing MLOps?, (3) What are the challenges practitioners face when adopting MLOps, and what solutions have been proposed to address them? Our findings emphasize emerging aspects of MLOps, i.e., compliance, governance, security, and quality, challenges and practices such as pipeline complexity, domain specific needs, automation tradeoffs, and scalability in distributed environments. With these, MLOps practices, roles, and activities evolved toward identifying new roles, collaboration and communication oriented practices, and frameworks that address new emerging and complex aspects of MLOps such as customization and complex infrastructures. We hope this study serves as a valuable resource for researchers and practitioners, guiding future advancements in MLOps adoption and innovation.

References

- [1] Kent Beck, Mike Beedle, Arie Van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. 2001. The agile manifesto. *Software Development* 9, 8 (2001), 28–35. <http://agilemanifesto.org>
- [2] Joseph Bradley, Rafi Kurlansik, Matt Thomson, and Niall Turbitt. 2023. The Big Book of MLOps. Databricks, (E-book).
- [3] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77.
- [4] Roland Croft, Yongzheng Xie, and Muhammad Ali Babar. 2022. Data preparation for software vulnerability prediction: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1044–1063.
- [5] IBM Data and AI Team. 2023. MLOps and the evolution of data science. Retrieved November, 2023 from <https://www.ibm.com/think/topics/mlops>
- [6] Nesara Dissanayake, Asangi Jayatilaka, Mansoor Zahedi, and M. Ali Babar. 2022. Software security patch management-A systematic literature review of challenges, approaches, tools and practices. *Information and Software Technology* 144 (2022), 106771.
- [7] Beyza Eken, Samodha Pallewatta, Nguyen Khoi Tran, Ayse Tosun, and Muhammad Ali Babar. 2025. Online appendix of A Multivocal Review of MLOps Practices, Challenges and Open Issues. <https://github.com/beken/MLOpsMLR>
- [8] Leonhard Faubel and Klaus Schmid. 2023. Review protocol: A systematic literature review of MLOps. (2023).
- [9] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106 (2019), 101–121.
- [10] IBM. 2022. IBM Global AI Adoption Index 2022. Retrieved February, 2024 from <https://www.ibm.com/downloads/documents/us-en/107a02e94a48f5c1>
- [11] Meenu Mary John, Helena Holmström Olsson, and Jan Bosch. 2021. Towards MLOps: A framework and maturity model. In *Proceedings of the 2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 1–8.
- [12] Barbara Kitchenham, Stuart Charters, et al. 2007. Guidelines for performing systematic literature reviews in software engineering: Technical report EBSE 2007-001. (2007).
- [13] Ask Berstad Kolltveit and Jingyue Li. 2022. Operationalizing machine learning models: A systematic literature review. In *Proceedings of the 1st Workshop on Software Engineering for Responsible AI*. 1–8.
- [14] Dominik Kreuzberger, Niklas Kühn, and Sebastian Hirschl. 2023. Machine learning operations (mlops): Overview, definition, and architecture. *IEEE Access* 11 (2023), 31866–31879.
- [15] Anderson Lima, Luciano Monteiro, and Ana Paula Furtado. 2022. MLOps: Practices, maturity models, roles, tools, and challenges-A systematic literature review. In *Proceedings of the International Conference on Enterprise Information Systems (ICEIS)*. 1 (2022), 308–320.
- [16] Lucy Ellen Lwakatare, Ivica Crnkovic, Ellinor Rånge, and Jan Bosch. 2020. From a data science driven process to a continuous delivery process for machine learning systems. In *Proceedings of the 21st International Conference on Product-Focused Software Process Improvement, PROFES 2020*. Springer, 185–201.
- [17] Tsakani Mboweni, Themba Masombuka, and Cyrille Dongmo. 2022. A systematic review of machine learning devops. In *Proceedings of the 2022 International Conference on Electrical, Computer and Energy Technologies (ICECET)*. IEEE, 1–6.
- [18] Fotis Psallidas, Megan Eileen Leszczynski, Mohammad Hossein Namaki, Avriela Floratou, Ashvin Agrawal, Konstantinos Karanasos, Subru Krishnan, Pavle Subotic, Markus Weimer, Yinghui Wu, et al. 2023. Demonstration of geyser: Provenance extraction and applications over data science scripts. In *Companion of the 2023 International Conference on Management of Data*. 123–126.
- [19] Khalid Salama, Jarek Kazmierczak, and Donna Schut. 2021. Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning. *Google Could White Paper* (2021).
- [20] Marius Schlegel and Kai-Uwe Sattler. 2023. MLflow2PROV: Extracting provenance from machine learning experiments. In *Proceedings of the 7th Workshop on Data Management for End-to-End Machine Learning*. 1–4.
- [21] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. In *Proceedings of the 29th International Conference on Neural Information Processing Systems 2* (2015), 2503–2511.
- [22] Monika Steidl, Michael Felderer, and Rudolf Ramler. 2023. The pipeline for the continuous development of artificial intelligence models-current state of research and practice. *Journal of Systems and Software* 199 (2023), 111615.
- [23] Monika Steidl, Rudolf Ramler, and Michael Felderer. 2024. The past, present, and future of research on the continuous development of AI. In *Proceedings of the 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 300–308.
- [24] Dezhan Tu, Yeye He, Weiwei Cui, Song Ge, Haidong Zhang, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. 2023. Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4991–5003.

- [25] Hironori Washizaki. 2024. Guide to the software engineering body of knowledge (SWEBOK guide), version 4.0. *IEEE Computer Society, Waseda University, Japan*. <http://www.swebok.org> *IEEE Computer Society*.
- [26] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. 1–10.

Received 11 June 2024; revised 5 June 2025; accepted 13 June 2025